HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO THIẾT KẾ
MÔN KIẾN TRÚC VÀ THIẾT KẾ PHẦN MỀM
Đề tài: Hệ thống quản lí phòng khám tư

Tên sinh viên	Nguyễn Nam Vũ
Mã sinh viên	B22DCCN916
Lớp	D22CNPM02

Giảng viên hướng dẫn: PGS.TS Nguyễn Mạnh Hùng

Hà Nội, 04/2026

Mục Lục

I. Hoạt động của từng module 3
a) Quản lí thông tin bệnh nhân 3
b) Khách hàng đặt lịch khám online 3
II. Thiết kế thực thể 5
III. Thiết kế cơ sở dữ liệu 7
a) Patient Service 7
b) Booking Service 7
c) Doctor Service 8
IV. Thiết kế kiến trúc 9
a) Patient Service 9
b) BookingService 10
c) DoctorService 12
V. Thiết kế cho từng module 14
a) Quản lí thông tin bệnh nhân 14
i. Thiết kế giao diện bên client/cho người dùng cuối 14
ii. Thiết kế biểu đồ lớp chi tiết cho mỗi bên/ứng dụng 17
1. Server là hộp đen 17
2. Client là hộp đen 17
iii. Thiết kế biểu đồ tuần tự hoạt động chi tiết cho mỗi bên/ứng dụng 21
1. Server là hộp đen 21
2. Client là hộp đen 23
b) Khách hàng đặt lịch khám online 26
i. Thiết kế giao diện bên client/cho người dùng cuối 26
ii. Thiết kế biểu đồ lớp chi tiết cho mỗi bên/ứng dụng 29
1. Server là hộp đen 29
2. Client là hộp đen 30
iii. Thiết kế biểu đồ tuần tự hoạt động chi tiết cho mỗi bên/ứng dụng 44
1. Server là hộp đen 45
2. Client là hộp đen 47
a) Patient Service (Đăng nhập với username và password lần lượt là mã và số điện thoại) ... 47
b) Booking Service (Đặt lịch khám) 49
c) Doctor Service(chọn khung giờ và bác sĩ làm việc trong khung giờ đó) 51

I. Hoạt động của từng module

a) Quản lí thông tin bệnh nhân

- Module quản lý thông tin bệnh nhân cho phép hệ thống thực hiện các thao tác truy xuất và quản lý hồ sơ bệnh nhân được lưu trong hệ thống. Người dùng có thể xem, tìm kiếm, thêm mới, cập nhật hay xóa hồ sơ của bệnh nhân.

- Luồng hoạt động chính xác gồm các bước :

1. Hệ thống hiển thị giao diện quản lý bệnh nhân.
2. Giao diện hiển thị danh sách bệnh nhân bao gồm: ID, mã bệnh nhân, họ tên, số điện thoại.
3. Có thể tìm kiếm bệnh nhân theo họ tên hoặc mã bệnh nhân.
4. Chọn "Thêm bệnh nhân" để tạo hồ sơ mới.
5. Hệ thống hiển thị form nhập thông tin bệnh nhân bao gồm: mã bệnh nhân, họ tên, số điện thoại.
6. Nhập thông tin và chọn "Thêm mới" để lưu.
7. Hệ thống (thông qua API Gateway) gọi Patient Service để lưu dữ liệu bệnh nhân mới vào cơ sở dữ liệu.
8. Chọn "Sửa" trên một dòng bệnh nhân cụ thể để cập nhật thông tin. Hệ thống hiển thị form cập nhật thông tin với các dữ liệu cũ (mã bệnh nhân, họ tên, số điện thoại) đã được điền sẵn. Người dùng tiến hành chỉnh sửa.
9. Xác nhận lưu, hệ thống gửi dữ liệu cập nhật qua API Gateway tới Patient Service. Patient Service tiến hành kiểm tra (validate) và lưu đè dữ liệu bệnh nhân mới vào cơ sở dữ liệu.
10. Chọn "Xóa" để xóa hồ sơ bệnh nhân.
11. Hệ thống gọi đến Patient Service để xác nhận và xóa vĩnh viễn dữ liệu bệnh nhân khỏi cơ sở dữ liệu.

b) Khách hàng đặt lịch khám online

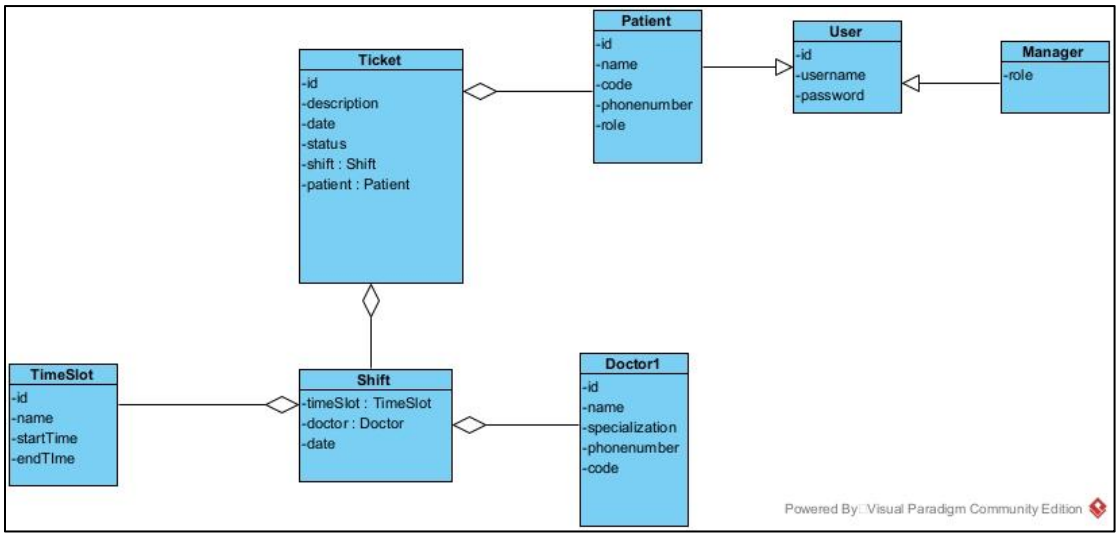
- Module Khách hàng đặt lịch khám online cho phép bệnh nhân tự thực hiện đăng ký lịch khám với các bác sĩ theo từng khung giờ/ca làm việc trong ngày. Người dùng có thể thao tác đặt lịch mới, xem lại lịch sử các phiếu khám, thao tác chỉnh sửa hoặc hủy các phiếu khám đang chờ xử lý.

- Luồng hoạt động chính gồm các bước:

1. Bệnh nhân truy cập giao diện hệ thống và tiến hành đăng nhập bằng Mã bệnh nhân và Số điện thoại.
2. Hệ thống xác thực, nếu thành công sẽ điều hướng sang giao diện Trang chủ (Dashboard).
3. Chọn chức năng "Đặt lịch khám". Hệ thống hiển thị giao diện chọn lịch trình.
4. Bệnh nhân chọn "Ngày khám" mong muốn và chọn "Ca khám" (ví dụ: Ca Sáng, Ca Chiều). Hệ thống lấy dữ liệu ca khám qua API Gateway (từ Doctor Service). Sau khi chọn ca, tiếp tục ấn "Tiếp theo".
5. Hệ thống truy xuất và hiển thị danh sách các Bác sĩ được phân công làm việc trong ca khám vừa chọn.
6. Bệnh nhân bấm chọn 1 Bác sĩ và ấn "Tiếp theo".
7. Hệ thống hiển thị Form xác nhận.
8. Bệnh nhân nhập thông tin "Mô tả triệu chứng" rồi bấm "Xác nhận".
9. Hệ thống lưu dữ liệu tạo Phiếu khám (Ticket) với trạng thái mặc định là "Tạo mới" (NEW) vào cơ sở dữ liệu, hiển thị giao diện phiếu với các thông tin người dùng vừa nhập để rà soát lần cuối.
10. Bệnh nhân rà soát thông tin lần cuối rồi bấm “ Đặt lịch ”.
11. Hệ thống lưu trạng thái của phiếu thành “Pending”.
12. Ngoài ra người dùng có thể xem lại các lần đặt lịch khám bệnh bằng chức năng "Lịch sử khám". Hệ thống hiển thị giao diện danh

sách tất cả các phiếu khám bệnh nhân đã đặt (gồm trạng thái PENDING, APPROVED, REJECTED).

II. Thiết kế thực thể



*Các lớp thực thể :

- User (người dùng) : chứa tài khoản, mật khẩu của người dùng hệ thống
- Manager(người dùng): là người dùng có quyền thay đổi thông tin bệnh nhân
- Patient (bệnh nhân) : chứa thông tin bệnh nhân
- Doctor (bác sĩ) : chứa thông tin bác sĩ
- TimeSlot (khung giờ): chứa khung giờ làm việc của bác sĩ
- Shift (ca làm việc) : chứa ca làm việc cụ thể của bác sĩ
- Ticket (lịch khám) : chứa thông tin về lịch khám của bệnh nhân

*Các mối quan hệ số lượng:

- Patient và Ticket: Một Patient có thể tạo nhiều Ticket, một Ticket chỉ thuộc về một Patient
- Ticket — Shift : Một Ticket chứa một Shift, một Shift có thể ở nhiều Ticket

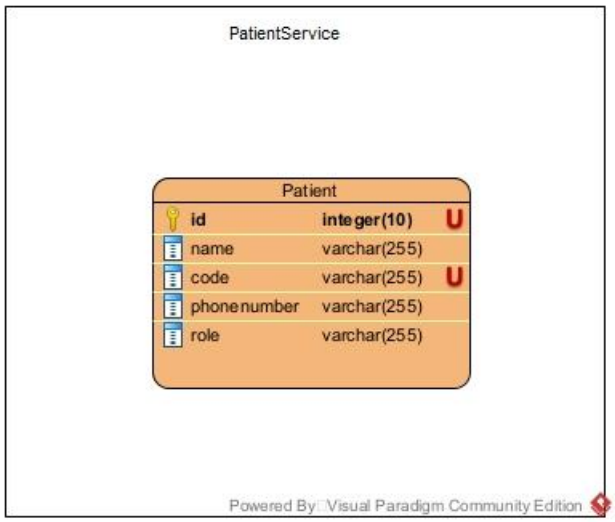
- Doctor và Shift: Một Doctor có thể phụ trách nhiều Shift, một Shift chỉ có một Doctor.
- TimeSlot và Shift: Một TimeSlot có thể được dùng cho nhiều Shift, một Shift chỉ thuộc một TimeSlot.
- Patient và User: Một Patient gắn với một tài khoản User để đăng nhập, và mỗi User đó thuộc về một Patient.
- Manager và User: Manager là một kiểu User (kế thừa từ User), tức mọi Manager đều là User nhưng có vai trò quản trị.

* Các quan hệ thành phần :

- Thực thể Shift chứa thông tin khung giờ từ TimeSlot để xác định thời gian làm việc. Do TimeSlot tồn tại độc lập và có thể được sử dụng lại, nên quan hệ giữa hai thực thể là aggregation.
- Thực thể Shift chứa thông tin bác sĩ từ Doctor để xác định người phụ trách ca làm. Do Doctor tồn tại độc lập và có thể tham gia nhiều ca làm việc khác nhau, nên quan hệ này là aggregation.
- Thực thể Ticket chứa thông tin ca làm từ Shift để xác định thời gian và bác sĩ tương ứng cho lịch khám. Do Shift tồn tại độc lập và có thể được sử dụng cho nhiều Ticket, nên quan hệ này là aggregation.
- Thực thể Ticket chứa thông tin bệnh nhân từ Patient để xác định người đặt lịch. Do Patient tồn tại độc lập và có thể có nhiều Ticket khác nhau, nên quan hệ này là aggregation.

III. Thiết kế cơ sở dữ liệu

a) Patient Service



id (integer): khóa chính dùng để định danh duy nhất mỗi bệnh nhân

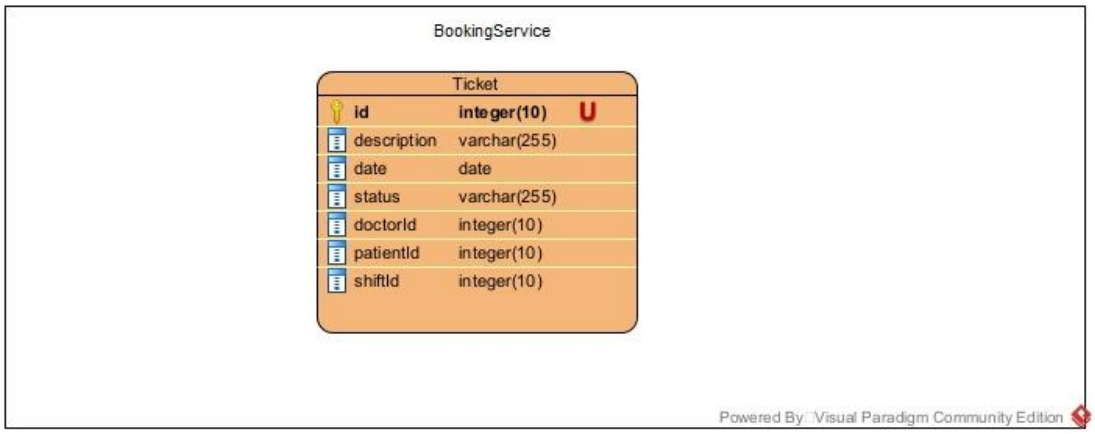
name (varchar): họ tên của bệnh nhân

code (varchar): mã bệnh nhân dùng làm username để đăng nhập

phonenumber (varchar): số điện thoại dùng làm password

role (varchar): thể hiện vai trò của bệnh nhân trong hệ thống

b) Booking Service



id (integer): khóa chính dùng để định danh duy nhất mỗi lịch khám

description (varchar): mô tả triệu chứng bệnh

date (date): ngày thực hiện lịch khám

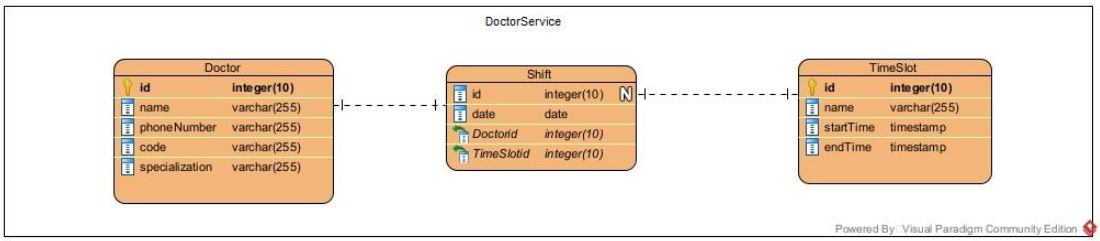
status (varchar): trạng thái của lịch khám (NEW, PENDING, APPROVED, CANCELLED)

doctorId (integer): mã định danh của bác sĩ được chọn khi đặt lịch

patientId (integer): mã định danh của bệnh nhân đặt lịch

shiftId (integer): mã định danh của ca làm việc mà lịch khám thuộc về

c) Doctor Service



- tblDoctor

id (integer): khóa chính dùng để định danh duy nhất mỗi bác sĩ

name (varchar): họ tên của bác sĩ

phoneNumber (varchar): số điện thoại dùng để liên hệ

code (varchar): mã định danh của bác sĩ trong hệ thống

specialization (varchar): chuyên ngành của bác sĩ

-tblShift:

id (integer): khóa chính dùng để định danh mỗi ca làm việc

date (date): ngày làm việc của ca

doctorId (integer): mã định danh của bác sĩ phụ trách ca làm

timeSlotId (integer): mã định danh của khung giờ áp dụng cho ca làm

-tblTimeSlot:

id (integer): khóa chính dùng để định danh mỗi khung giờ

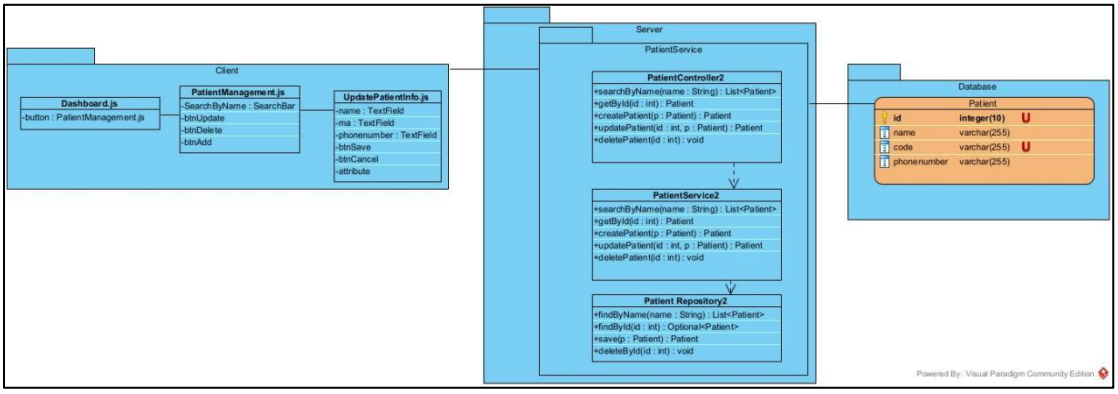
name (varchar): tên khung giờ (ca sáng, ca chiều, ca tối)

startTime (timestamp): thời gian bắt đầu của khung giờ

endTime (timestamp): thời gian kết thúc của khung giờ

IV. Thiết kế kiến trúc

a) Patient Service



*Client

- Dashboard.js

BtnPatientManagement : nút điều hướng sang màn quản lý bệnh nhân PatientManagement.js

- PatientManagement.js

searchByName(name): gọi API tìm bệnh nhân theo tên

btnAdd (sự kiện click): mở form thêm bệnh nhân

btnUpdate (sự kiện click): mở form sửa cho bệnh nhân đã chọn

btnDelete (sự kiện click): xóa bệnh nhân đã chọn

renderTable(list): hiển thị danh sách bệnh nhân lên bảng

- UpdatePatientInfo.js

name: khung nhập tên bệnh nhân để sửa

code: khung nhập mã bệnh nhân để sửa

phonenumber: khung nhập số điện thoại bệnh nhân để sửa

submitUpdate(): gửi dữ liệu cập nhật về server

cancelUpdate(): hủy thao tác và quay lại màn danh sách

*Server (PatientService)

- PatientController:

searchByName(name): nhận request tìm theo tên, trả danh sách bệnh nhân

getById(id): lấy chi tiết 1 bệnh nhân theo id

createPatient(p): tạo bệnh nhân mới

updatePatient(id, p): cập nhật thông tin bệnh nhân theo id

deletePatient(id): xóa bệnh nhân theo id

- PatientService

searchByName(name): xử lý nghiệp vụ tìm kiếm theo tên

getById(id): kiểm tra tồn tại và trả bệnh nhân theo id

createPatient(p): kiểm tra ràng buộc, lưu mới

updatePatient(id, p): kiểm tra ràng buộc, cập nhật dữ liệu

deletePatient(id): kiểm tra rồi xóa bệnh nhân

-PatientRepository

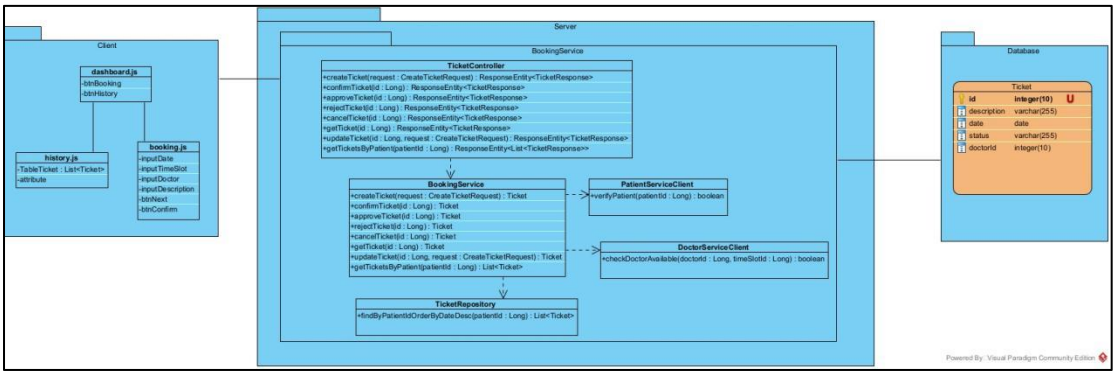
findByNameContaining(name): truy vấn danh sách theo tên gần đúng

findById(id): truy vấn 1 bệnh nhân theo id

save(p): insert hoặc update bệnh nhân

deleteById(id): xóa bệnh nhân theo id

b) BookingService



*Client

- login.js

inputUsername: ô nhập username (Patient.code)

inputPassword: ô nhập password (Patient.phonenumber)

subLogin: đăng nhập và kiểm tra username,password

- dashboard.js

btnBooking: điều hướng sang chức năng Booking

btnHistory : điều hướng sang chức năng History

- booking.js

inputDate: ô nhập ngày

inputTimeSlot: lựa chọn khung giờ

inputDoctor: chọn bác sĩ trong với khung giờ đã chọn

inputDesc: mô tả triệu chứng bệnh

Confirm để gửi yêu cầu tạo ticket.

- history.js

Ý nghĩa: màn xem lịch sử, hiển thị bảng tableTicket gồm listTicket và các thuộc tính của ticket.

*Server

- TicketController

createTicket(request: CreateTicketRequest): tạo ticket mới.

confirmTicket(id: Long): xác nhận ticket.

approveTicket(id: Long): duyệt ticket.

rejectTicket(id: Long): từ chối ticket.

cancelTicket(id: Long): hủy ticket.

getTicket(id: Long): lấy chi tiết 1 ticket.

getTicketsByPatient(patientId: Long): lấy danh sách ticket theo bệnh nhân.

updateTicket(id: Long, request: CreateTicketRequest): cập nhật ticket.

- BookingService

Các hàm tương ứng với controller để xử lí logic

createTicket, confirmTicket, approveTicket, rejectTicket, cancelTicket, getTicket, getTicketsByPatient, updateTicket.

- PatientServiceClient: verifyPatient(patientId: Long): boolean

Vai trò: kiểm tra bệnh nhân có tồn tại trước khi tạo/cập nhật phiếu khám.

- DoctorServiceClient: checkDoctorAvailable(doctorId: Long, timeSlotId: Long): boolean

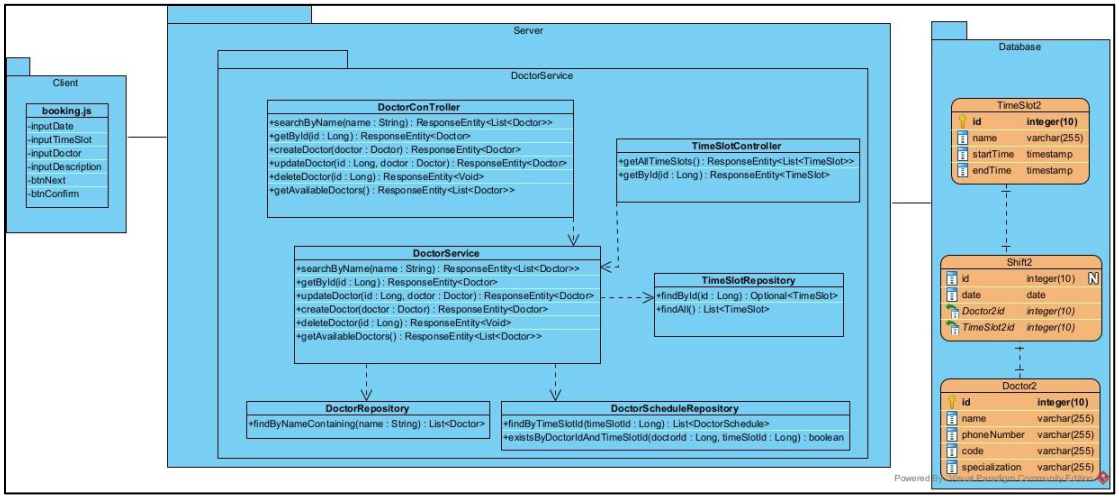
Vai trò: kiểm tra bác sĩ có trống theo khung giờ trước khi đặt lịch.

- TicketRepository

findByPatientIdOrderByDateDesc(patientId: Long): lấy ticket theo bệnh nhân, sắp xếp ngày giảm dần.

Các hàm còn lại như findById, save, deleteById được kế thừa từ JpaRepository.

c) DoctorService



*Client

- booking.js : giao diện bệnh nhân dùng để đặt lịch khám.

inputDate: nhập ngày khám.

inputTimeSlot: chọn ca khám.

inputDoctor: chọn bác sĩ theo ca đã chọn.

inputDescription: nhập mô tả triệu chứng.

btnNext: chuyển bước trong luồng đặt lịch.

btnConfirm: gửi yêu cầu xác nhận đặt lịch.

*Server (DoctorService)

- DoctorController: lớp REST API cho nghiệp vụ bác sĩ.

searchByName(name): tìm bác sĩ theo tên.

getById(id): lấy thông tin bác sĩ theo id.

createDoctor(doctor): tạo bác sĩ mới.

updateDoctor(id, doctor): cập nhật thông tin bác sĩ.

deleteDoctor(id): xóa bác sĩ.

getAvailableDoctors(): lấy danh sách bác sĩ đang available.

- TimeSlotController: lớp REST API cho ca khám.

getAllTimeSlots(): lấy toàn bộ ca khám.

getById(id): lấy chi tiết một ca khám.

- DoctorService: tầng nghiệp vụ trung tâm, xử lý logic trước khi truy cập repository.

searchByName(name)

getById(id)

createDoctor(doctor)

updateDoctor(id, doctor)

deleteDoctor(id)

getAvailableDoctors()

Ngoài ra còn phối hợp truy vấn lịch trực theo ca (qua DoctorScheduleRepository) để hỗ trợ luồng chọn bác sĩ theo time slot.

- DoctorRepository: truy cập dữ liệu Doctor.

findByNameContaining(name): tìm bác sĩ theo tên gần đúng.

- TimeSlotRepository:truy cập dữ liệu TimeSlot.

findById(id): tìm kiếm TimeSlot theo id

findAll(): lấy tất cả TimeSlot

- DoctorScheduleRepository: truy cập dữ liệu lịch phân ca bác sĩ.

findByTimeSlotId(timeSlotId): lấy lịch theo ca để suy ra danh sách bác sĩ.

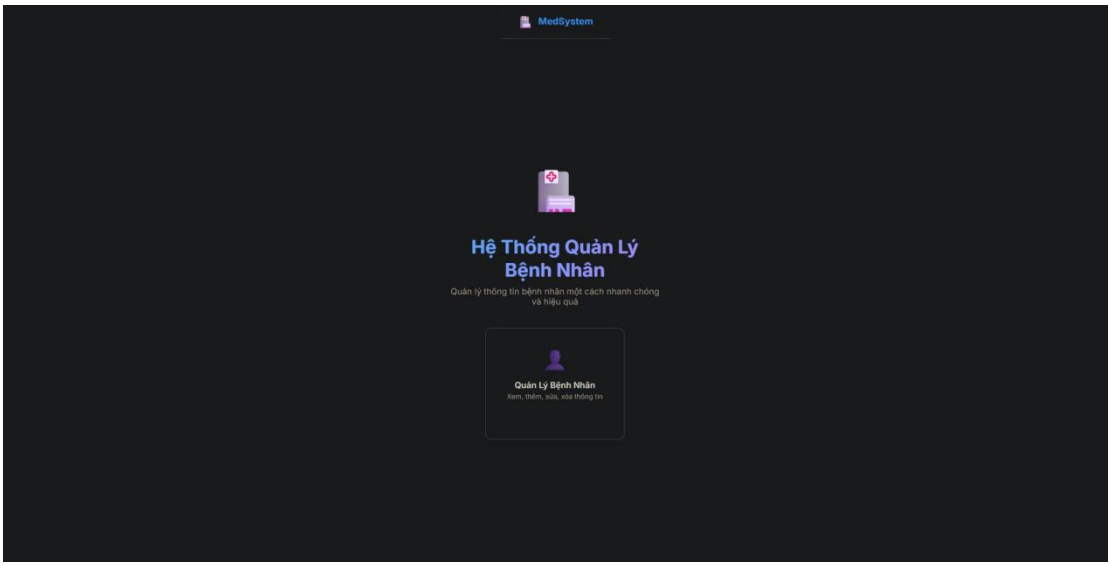
existsByDoctorIdAndTimeSlotId(doctorId, timeSlotId): kiểm tra bác sĩ có làm ca đó không.

V. Thiết kế cho từng module

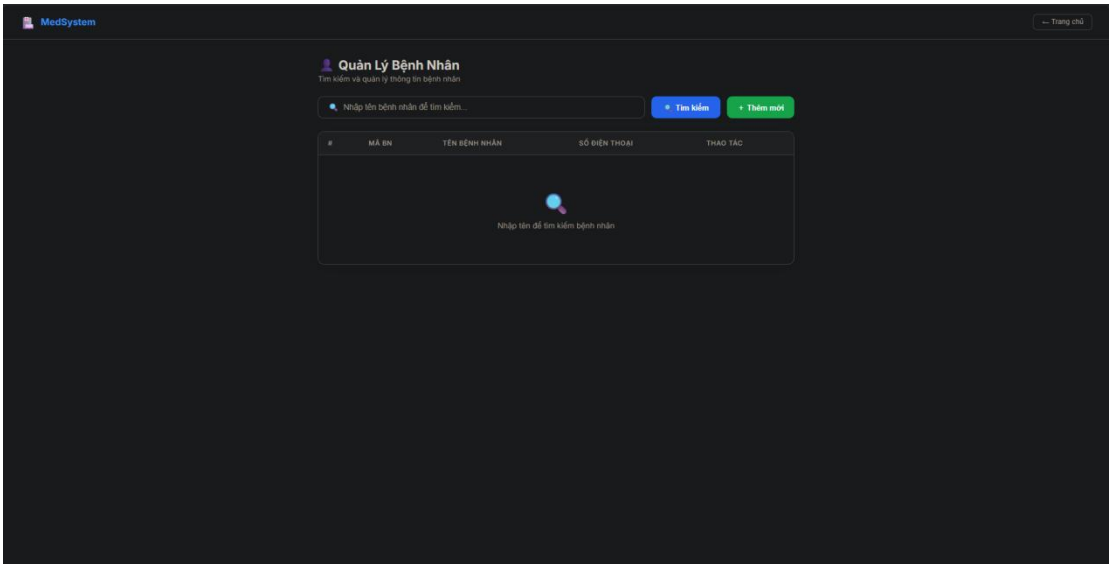
a) Quản lí thông tin bệnh nhân

i. Thiết kế giao diện bên client/cho người dùng cuối

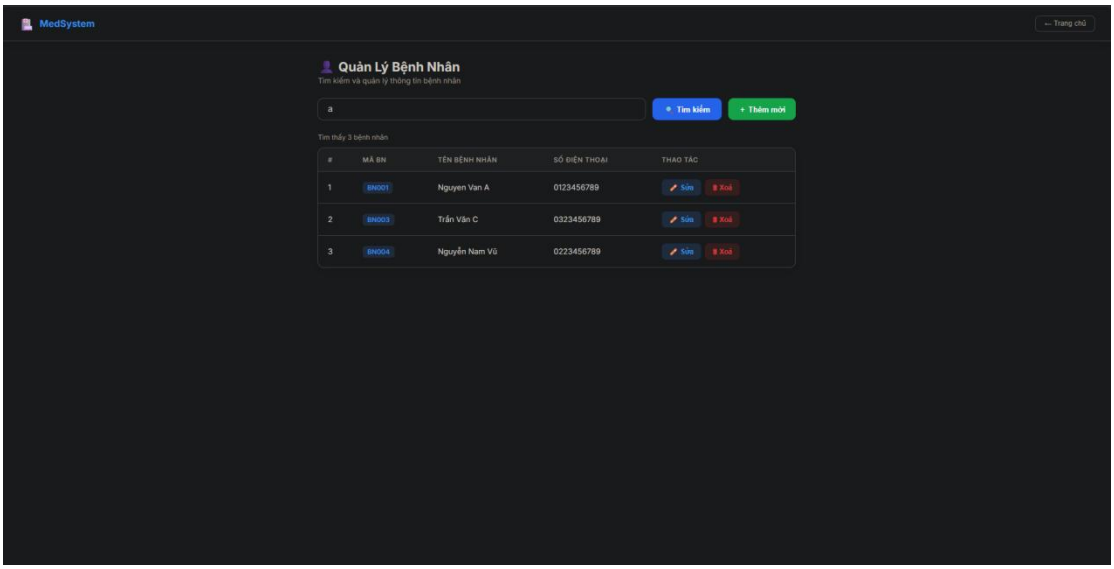
- Giao diện trang chủ



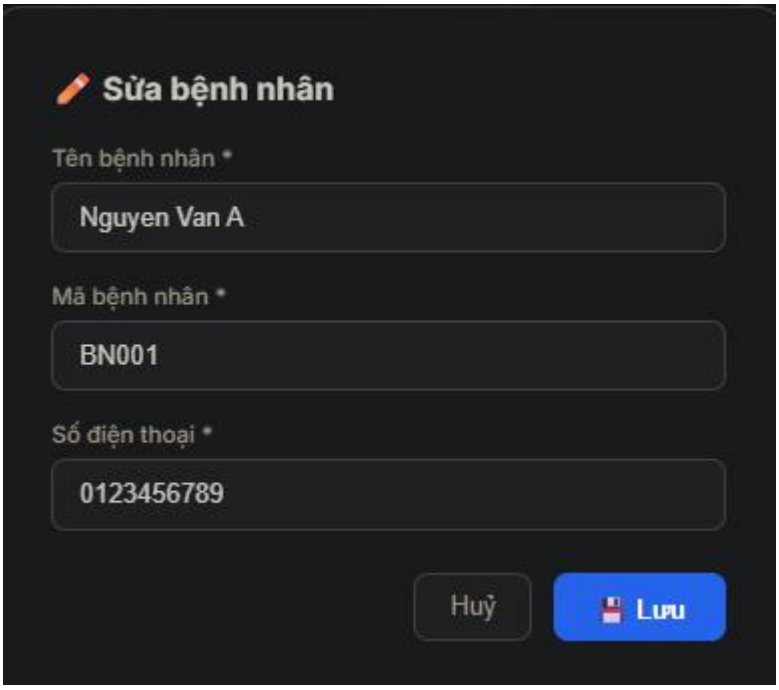
- Giao diện quản lí thông tin bệnh nhân



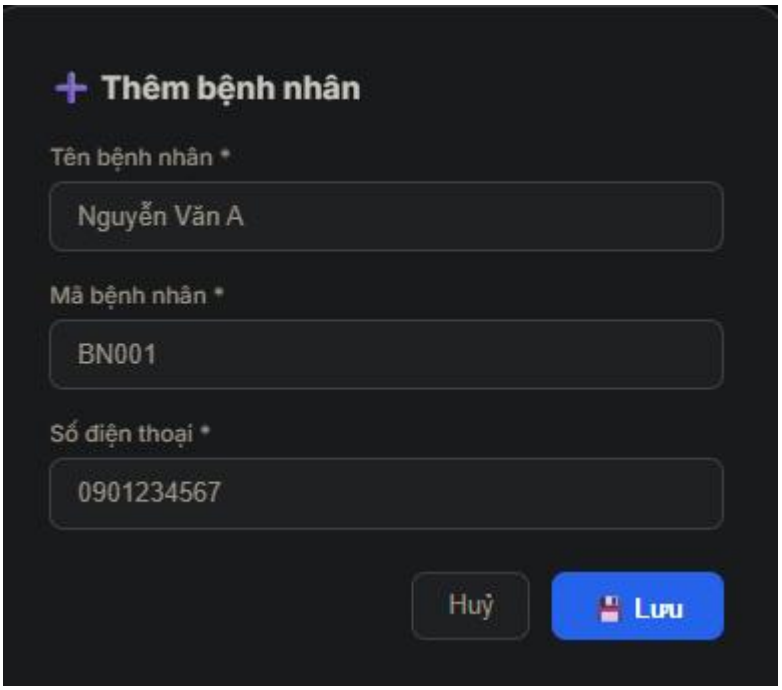
- Giao diện tìm kiếm



- Giao diện sửa thông tin 1 bệnh nhân

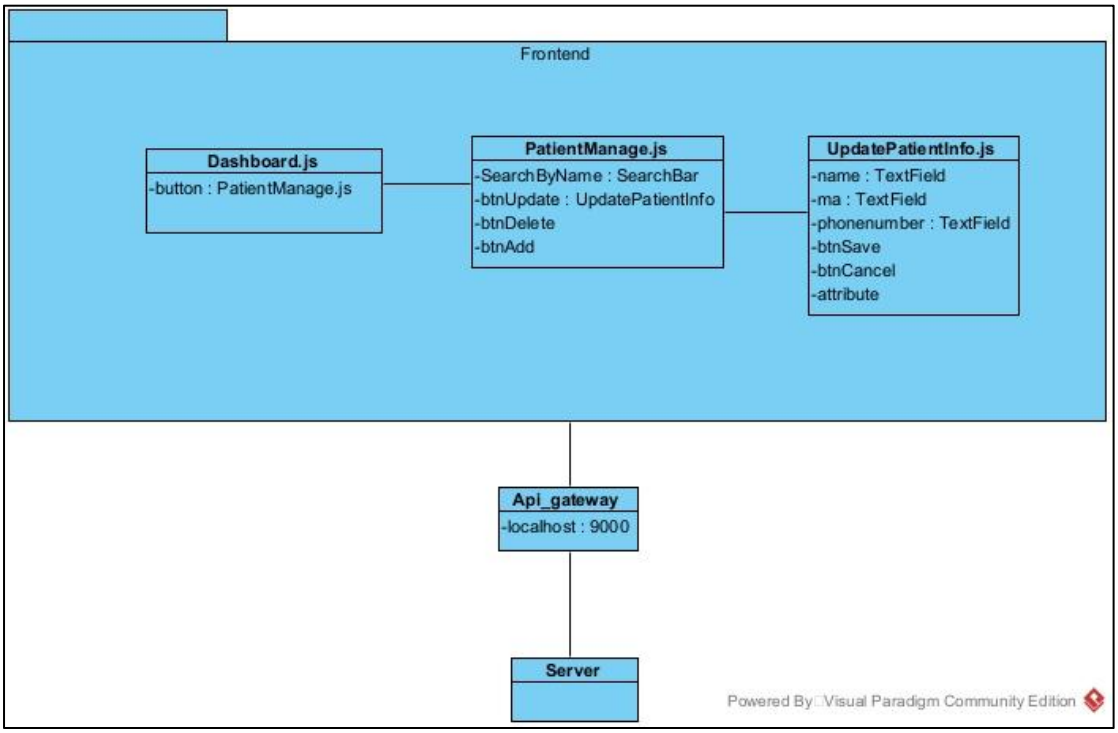


- Giao diện thêm thông tin bệnh nhân



ii. Thiết kế biểu đồ lớp chi tiết cho mỗi bên/ứng dụng

1. Server là hộp đen

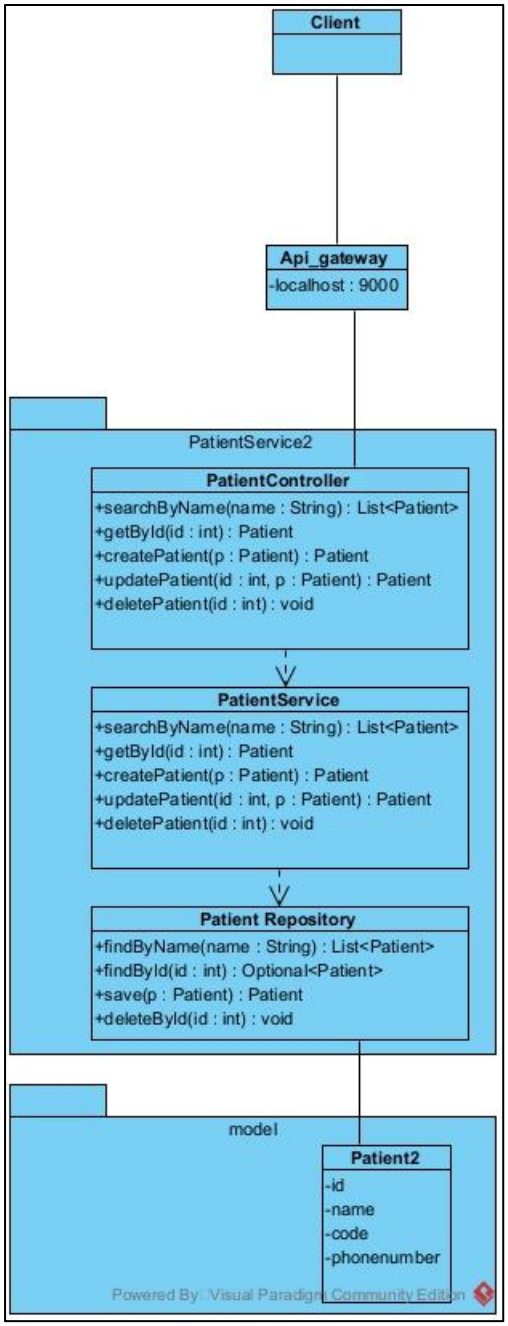


Luồng hoạt động

- Người dùng thao tác trên Dashboard.
- Chuyển sang PatientManagement để tìm kiếm, thêm, sửa hoặc xóa bệnh nhân.
- Nếu cần sửa/thêm, hệ thống mở UpdatePatientInfo.
- Các yêu cầu từ frontend đi qua API Gateway.
- API Gateway chuyển request xuống Server để xử lý.

2. Client là hộp đen

*Patient Service



Service này phụ trách toàn bộ nghiệp vụ quản lý bệnh nhân, gồm tìm kiếm, xem chi tiết, thêm mới, cập nhật và xóa bệnh nhân.

- Luồng tổng quát:

Client gửi request qua API Gateway.

API Gateway chuyển request đến PatientController.

PatientController gọi PatientService.

PatientService xử lý nghiệp vụ và gọi PatientRepository.

PatientRepository thao tác với database bằng entity Patient. Kết quả được trả ngược lại cho Client.

1. Client

- Client là giao diện người dùng, nơi thực hiện các thao tác với bệnh nhân. Frontend gọi xuống hệ thống thông qua API Gateway ở cổng localhost:9000.
- Các chức năng chính: Tìm kiếm bệnh nhân theo tên, Xem chi tiết bệnh nhân theo id, Thêm bệnh nhân mới, Cập nhật thông tin bệnh nhân, Xóa bệnh nhân

2. API Gateway: đóng vai trò trung gian nhận request từ Client và chuyển tiếp đến Patient Service.

3. PatientController

- Đây là lớp REST API của Patient Service, nhận request từ gateway rồi gọi xuống service xử lý.
- Các hàm:

searchByName(name: String): List<Patient>

Tìm danh sách bệnh nhân có tên phù hợp với từ khóa.

getById(id: int): Patient

Lấy ra một bệnh nhân theo id.

createPatient() : Tạo mới một bệnh nhân.

updatePatient():Cập nhật thông tin bệnh nhân theo id.

deletePatient(): Xóa bệnh nhân theo id.

4. PatientService

- Đây là tầng nghiệp vụ trung gian, xử lý logic trước khi làm việc với database.
- Các hàm:

searchByName(): Xử lý tìm kiếm bệnh nhân theo tên.

getById(): Kiểm tra và lấy bệnh nhân theo id.

createPatient(): Kiểm tra dữ liệu rồi lưu bệnh nhân mới.

updatePatient(): Cập nhật thông tin bệnh nhân.

deletePatient(): Kiểm tra rồi xóa bệnh nhân.

5. PatientRepository

- Đây là lớp truy cập dữ liệu, làm việc trực tiếp với database.
- Các hàm:

findByName(): Tìm bệnh nhân theo tên.

findById(): Tìm bệnh nhân theo id.

save(): Lưu bệnh nhân mới hoặc cập nhật bệnh nhân cũ.

deleteById(): Xóa bệnh nhân theo id.

6. Model - Patient

Đây là thực thể bệnh nhân dùng để lưu dữ liệu.

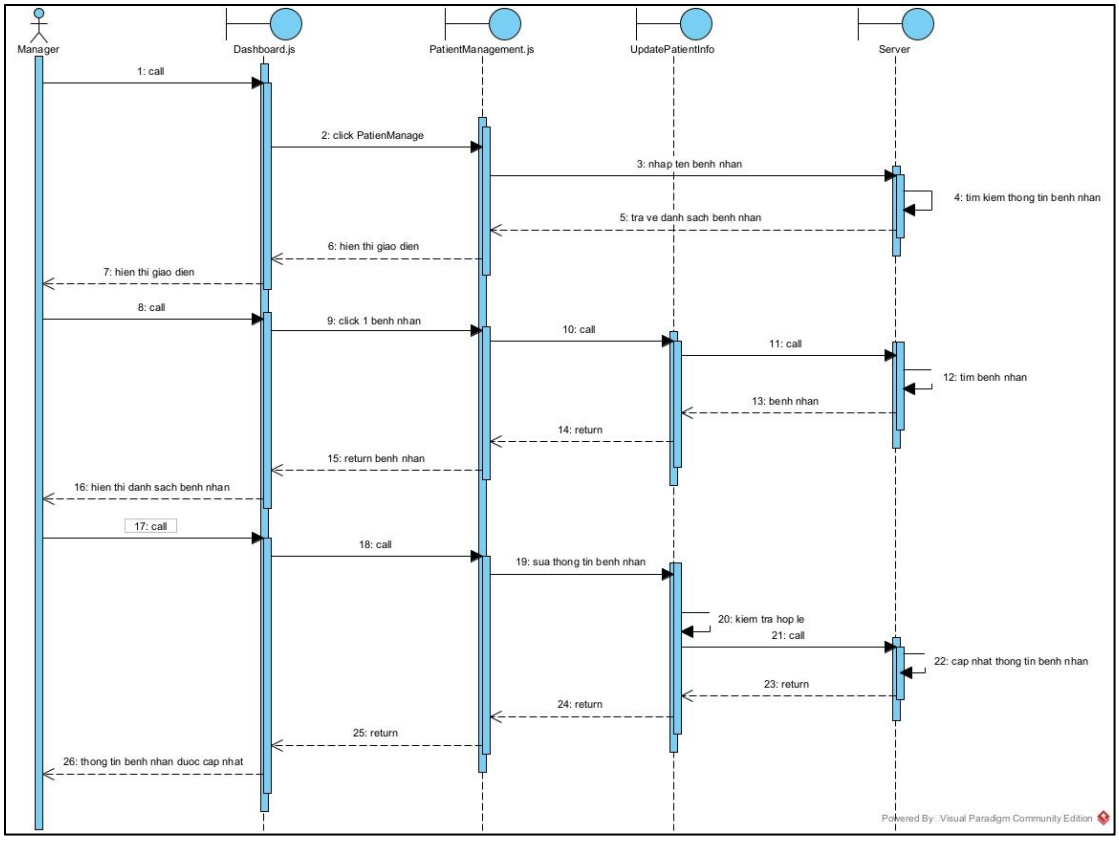
Thuộc tính:id, name, code, phonenumber

iii. Thiết kế biểu đồ tuần tự hoạt động chi tiết cho mỗi bên/ứng dụng

Kịch bản chuẩn:

1. Quản lý đăng nhập thành công
2. Giao diện dashboard quản lí hiện ra gồm nút quản lí thông tin bệnh nhân
3. Quản lý click vào nút quản lí thông tin bệnh nhân
4. Giao diện quản lí thông tin bệnh nhân hiện ra gồm thanh tìm kiếm bệnh nhân
5. Quản lí nhập "a" vào thanh tìm kiếm
6. Giao diện hiện ra danh sách các bệnh nhân có tên gần đúng với kết quả tìm kiếm
7. Quản lí click vào bệnh nhân tên "Nguyễn Văn A"
8. Giao diện thông tin chi tiết của bệnh nhân hiện ra
9. Quản lí thay đổi tên thành "Nguyễn Văn An" và bấm xác nhận.
10. Hệ thống thông báo "Thay đổi thông tin thành công " và quay lại giao diện danh sách các bệnh nhân.

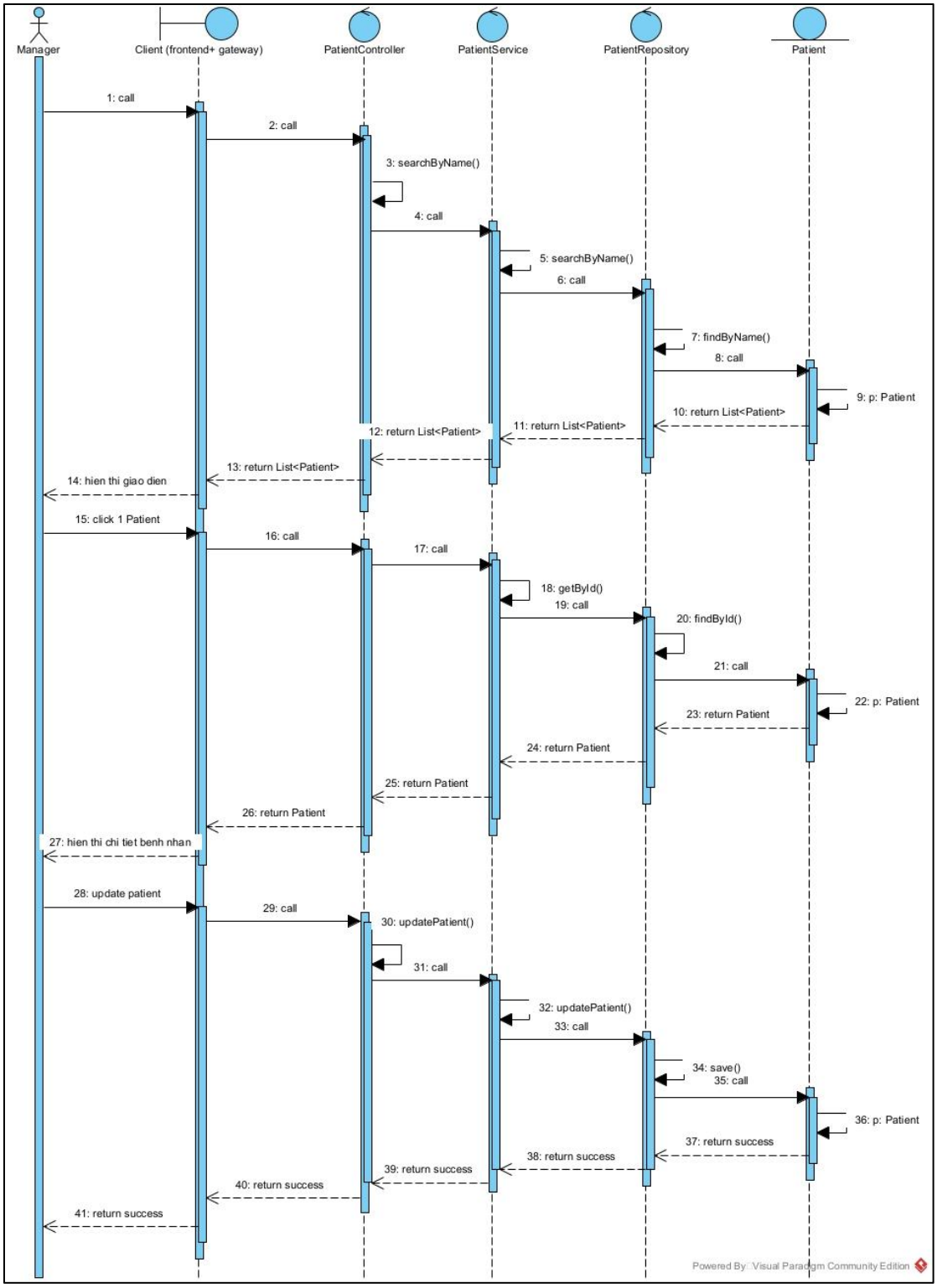
1. Server là hộp đen



1. Manager gửi yêu cầu (call) đến Dashboard.
2. Tại Dashboard, người dùng click vào chức năng PatientManage.
3. PatientManagement nhận input tên bệnh nhân cần tìm.
4. Server thực hiện tìm kiếm thông tin bệnh nhân theo tên.
5. Server trả về danh sách bệnh nhân phù hợp.
6. PatientManagement hiển thị giao diện danh sách bệnh nhân.
7. Dashboard hiển thị giao diện cho Manager.
8. Manager tiếp tục gửi thao tác (call) trên Dashboard.
9. Người dùng chọn (click) một bệnh nhân cụ thể.
10. PatientManagement gọi sang module UpdatePatientInfo.
11. UpdatePatientInfo gửi request đến Server.
12. Server thực hiện tìm bệnh nhân theo ID.
13. Server trả về thông tin chi tiết bệnh nhân.
14. UpdatePatientInfo nhận dữ liệu và trả về cho PatientManagement.
15. PatientManagement trả dữ liệu bệnh nhân về Dashboard.

16. Dashboard hiển thị danh sách/thông tin bệnh nhân cho Manager.
17. Manager thực hiện thao tác chỉnh sửa (call).
18. Dashboard gửi yêu cầu đến PatientManagement.
19. PatientManagement gửi yêu cầu sửa thông tin bệnh nhân sang UpdatePatientInfo.
20. UpdatePatientInfo kiểm tra tính hợp lệ của dữ liệu.
21. Nếu hợp lệ, gửi request cập nhật đến Server.
22. Server thực hiện cập nhật thông tin bệnh nhân trong hệ thống.
23. Server trả kết quả cập nhật về cho UpdatePatientInfo.
24. UpdatePatientInfo trả kết quả về cho PatientManagement.
25. PatientManagement trả kết quả về Dashboard.
26. Dashboard hiển thị thông báo: thông tin bệnh nhân đã được cập nhật thành công.

2. Client là hộp đen



1. Manager gửi yêu cầu (call) đến Client (frontend + gateway).
2. Client gửi request đến PatientController.
3. PatientController xử lý yêu cầu tìm kiếm searchByName().
4. PatientController gọi sang PatientService.
5. PatientService tiếp tục xử lý searchByName().

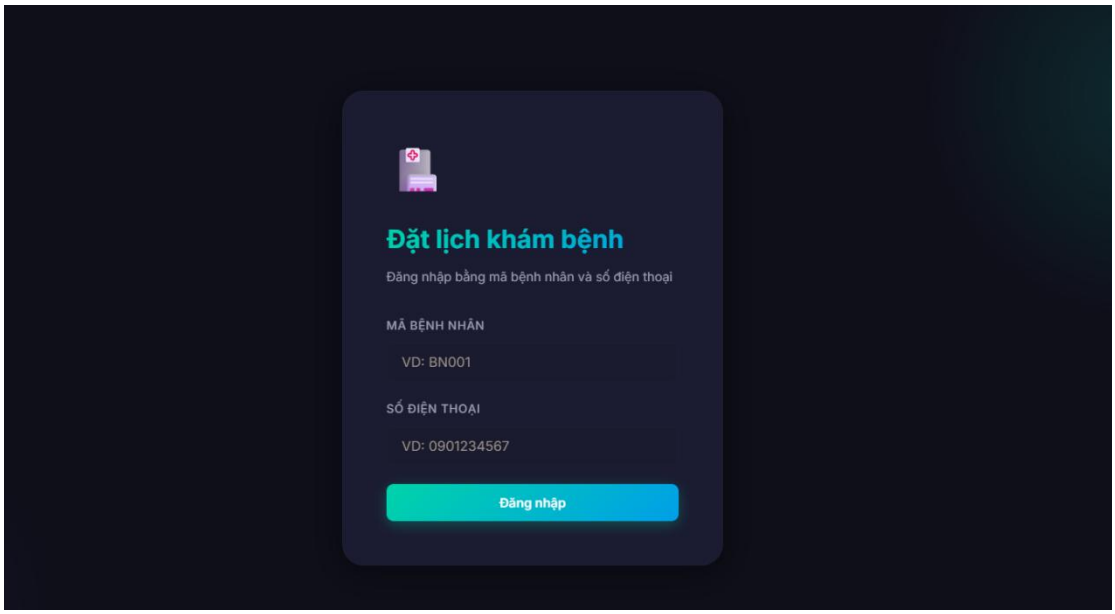
6. PatientService gọi đến PatientRepository.
7. PatientRepository thực hiện findByName() trên dữ liệu.
8. PatientRepository truy vấn đến thực thể Patient.
9. Trả về danh sách bệnh nhân (List<Patient>).
10. PatientRepository trả kết quả về PatientService.
11. PatientService trả danh sách bệnh nhân về PatientController.
12. PatientController trả danh sách về Client.
13. Client nhận dữ liệu và chuẩn bị hiển thị.
14. Giao diện hiển thị danh sách bệnh nhân cho Manager.
15. Manager chọn (click) một bệnh nhân.
16. Client gửi request đến PatientController.
17. PatientController gọi sang PatientService.
18. PatientService xử lý getById().
19. PatientService gọi PatientRepository.
20. PatientRepository thực hiện findById().
21. PatientRepository truy vấn đến thực thể Patient.
22. Trả về thông tin chi tiết bệnh nhân.
23. PatientRepository trả dữ liệu về PatientService.
24. PatientService trả dữ liệu về PatientController.
25. PatientController trả dữ liệu về Client.
26. Client nhận dữ liệu bệnh nhân.
27. Giao diện hiển thị chi tiết bệnh nhân cho Manager.
28. Manager thực hiện cập nhật thông tin bệnh nhân.
29. Client gửi request đến PatientController.
30. PatientController gọi hàm updatePatient().
31. PatientController gọi sang PatientService.
32. PatientService xử lý updatePatient().
33. PatientService gọi PatientRepository.
34. PatientRepository thực hiện save().

35. PatientRepository cập nhật dữ liệu vào thực thể Patient.
36. Trả về đối tượng bệnh nhân đã cập nhật.
37. PatientRepository trả kết quả thành công về PatientService.
38. PatientService trả kết quả thành công về PatientController.
39. PatientController trả kết quả về Client.
40. Client nhận phản hồi thành công.
41. Giao diện thông báo cập nhật bệnh nhân thành công cho Manager.

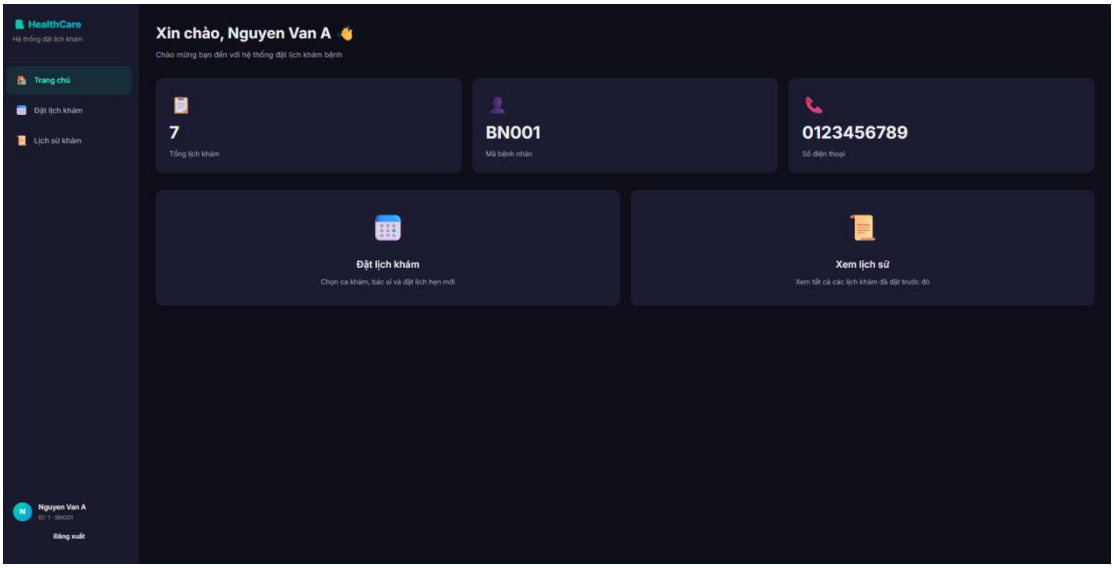
b) Khách hàng đặt lịch khám online

i. Thiết kế giao diện bên client/cho người dùng cuối

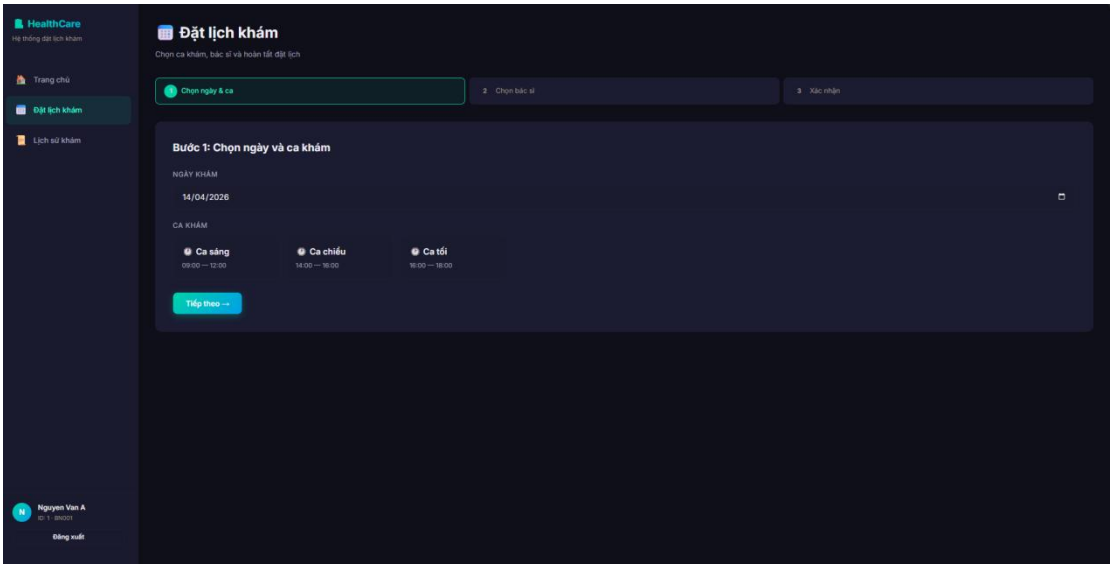
- Giao diện Login

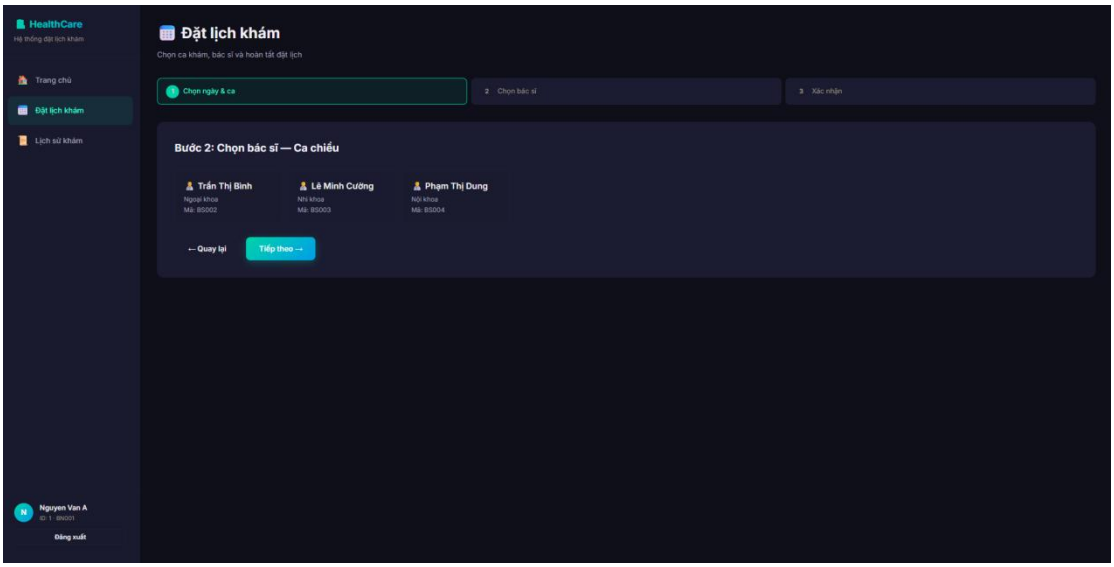


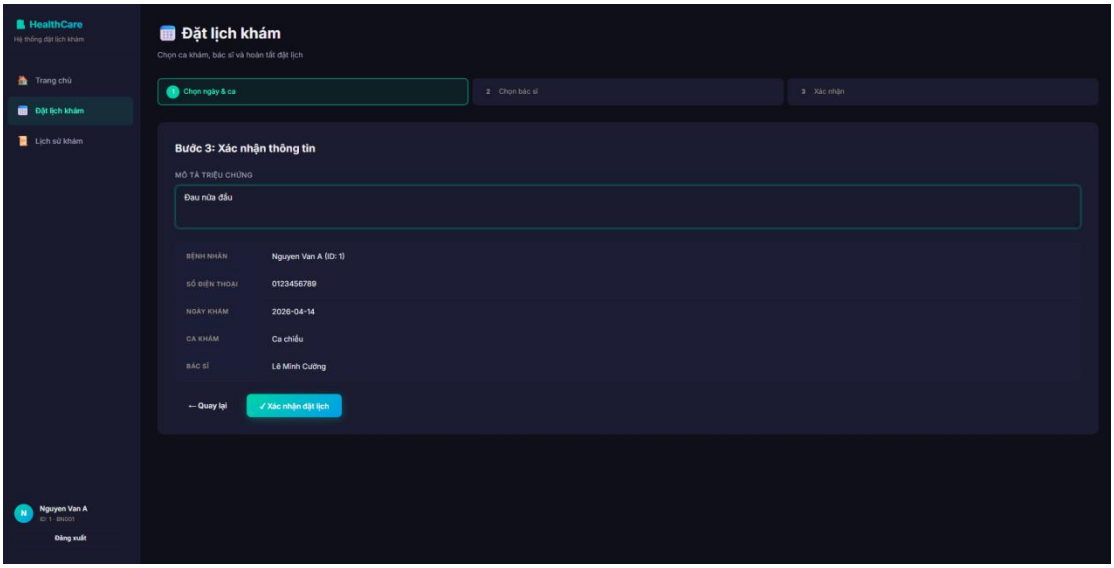
- Giao diện trang chủ


HealthCare
Hệ thống đặt lịch khám
Trang chủ
Đặt lịch khám
Lịch sử khám
Nguyen Van A
ID: 1 · BN001
Đăng xuất
Xin chào, Nguyen Van A 👋
Chào mừng bạn đến với hệ thống đặt lịch khám bệnh
7
Tổng lịch khám
BN001
Mã bệnh nhân
0123456789
Số điện thoại
Đặt lịch khám
Chọn ca khám, bác sĩ và đặt lịch hẹn mới
Xem lịch sử
Xem tất cả các lịch khám đã đặt trước đó

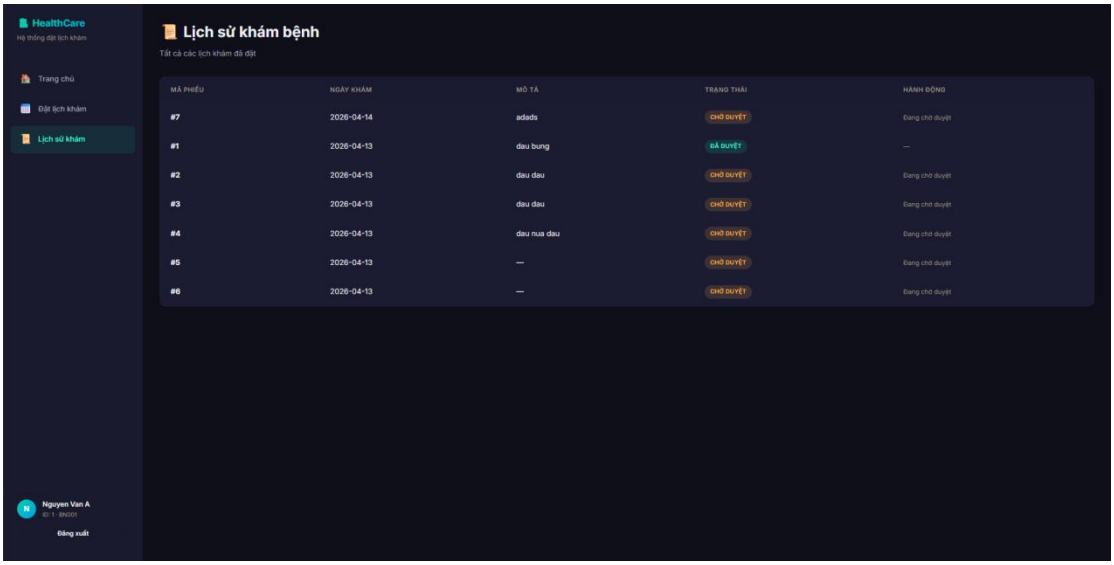
- Giao diện đặt lịch khám





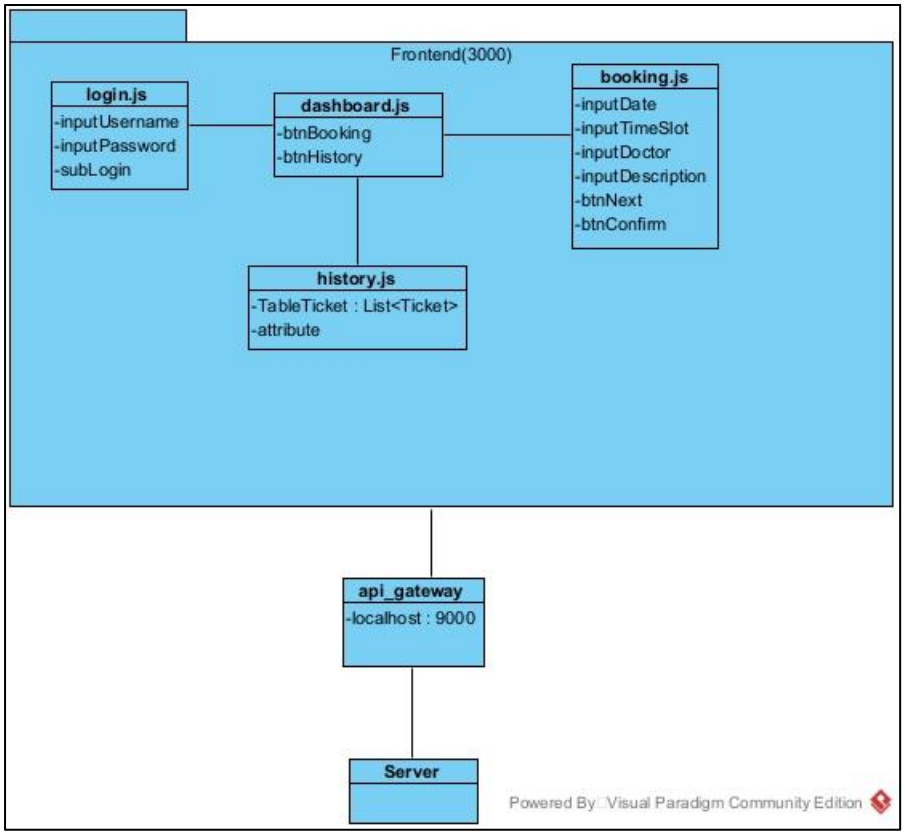


- Giao diện lịch sử khám



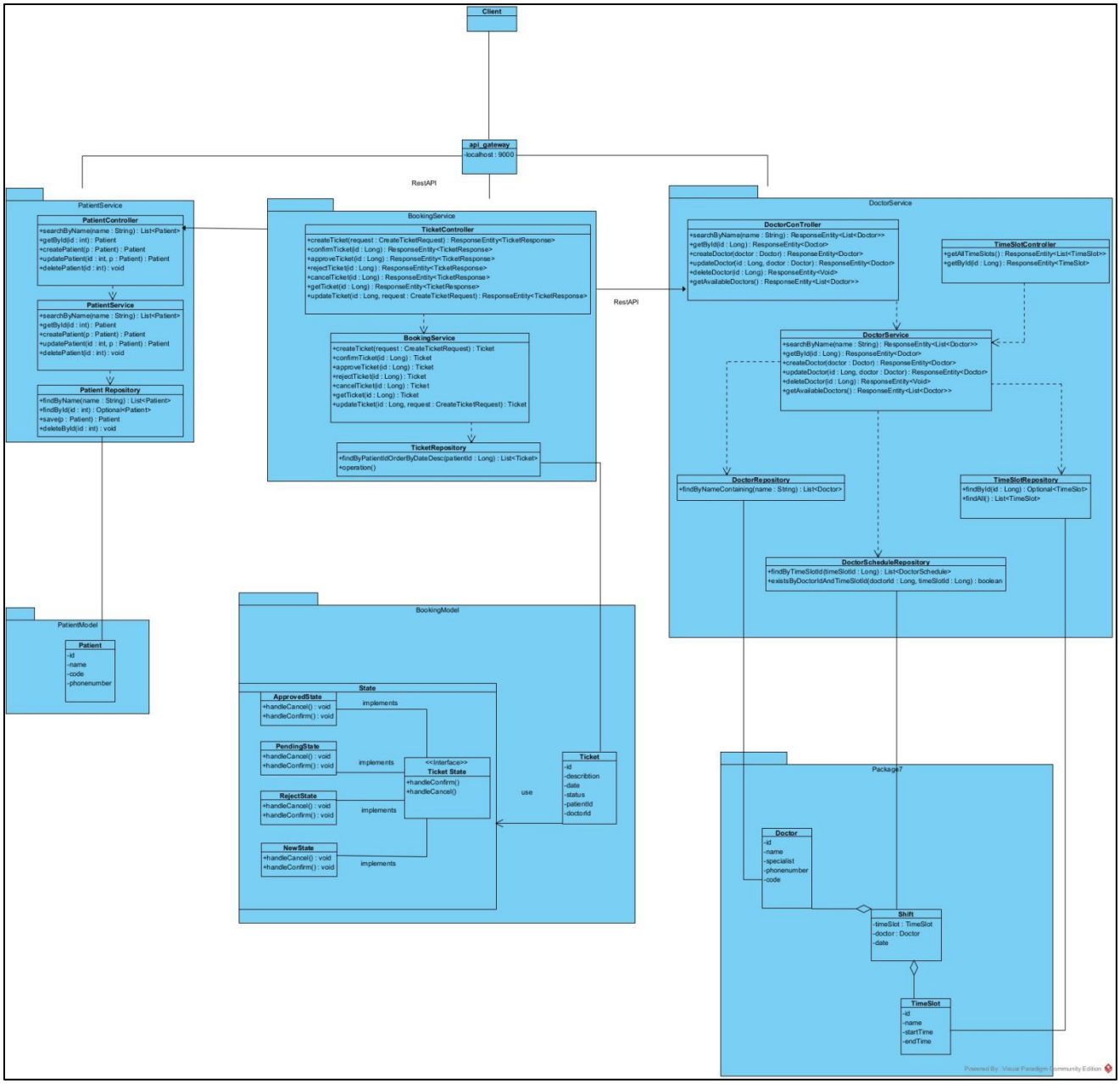
ii. Thiết kế biểu đồ lớp chi tiết cho mỗi bên/ứng dụng

1. Server là hộp đen



2. Client là hộp đen

   i. Tổng hợp



Luồng hoạt động tổng quát

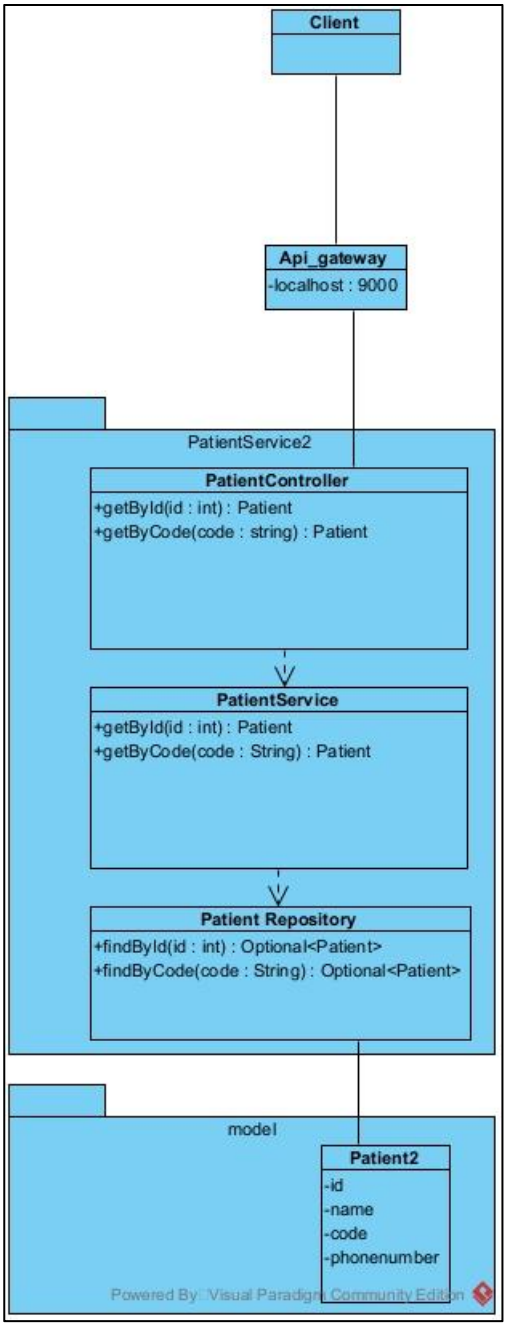
1. Người dùng đăng nhập vào hệ thống từ frontend.

2. Tại màn hình đặt lịch khám, người dùng tự chọn ngày khám trên ô nhập ngày. Ngày này là dữ liệu do người dùng nhập trực tiếp ở giao diện, không cần gọi service để lấy.

3. Sau khi người dùng chọn ngày khám, hệ thống gửi request qua API Gateway đến DoctorService để lấy danh sách ca khám (TimeSlot) còn hiển thị cho bệnh nhân.

4. Khi người dùng chọn một ca khám, hệ thống tiếp tục gửi request qua DoctorService để lấy danh sách bác sĩ làm việc trong ca đó.
5. Người dùng chọn bác sĩ phù hợp và nhập mô tả triệu chứng, nhấn xác nhận.
6. Frontend gửi request qua API Gateway đến BookingService để tạo ticket đặt lịch.
7. BookingService tạo ticket mới và lưu xuống database, Ticket sẽ có status là New.
8. Frontend nhận được thông tin và hiện thị Ticket có trạng thái New lên để người dùng xác nhận lần cuối.
9. Ngươi dùng nhấn xác nhận.
10. Frontend gửi thông tin đến Booking Service để thay đổi trạng thái phiếu thành pending
11. Hệ thống trả kết quả về frontend và hiển thị đặt lịch thành công.
12. Người dùng có thể chuyển sang màn lịch sử để xem ticket vừa tạo.

ii. PatientService



Diễn giải:

Trong chức năng đặt lịch khám, Patient Service có vai trò xác thực và cung cấp thông tin bệnh nhân (theo id hoặc code) để Booking Service kiểm tra bệnh nhân hợp lệ trước khi tạo phiếu khám.

*Client: Là giao diện người dùng hoặc service gọi đến để lấy thông tin bệnh nhân khi đặt lịch.

Client không truy cập trực tiếp PatientService, mà đi qua API Gateway.

*API Gateway (localhost:9000)

Là điểm vào duy nhất của request.

Nhận request từ Client rồi định tuyến đến PatientService2.

*PatientService (server-side)

Bao gồm 3 lớp theo kiến trúc phân tầng:

- PatientController :

getById(id:int): Patient

getByCode(code:String): Patient

Vai trò: nhận request HTTP, gọi xuống service và trả kết quả cho client.

- PatientService:

getById(id:int): Patient

getByCode(code:String): Patient

Vai trò: xử lý nghiệp vụ trung gian, kiểm tra dữ liệu trước khi truy cập

- PatientRepository

findById(id:int): Optional<Patient>

findByCode(code:String): Optional<Patient>

Vai trò: truy vấn dữ liệu bệnh nhân trong database.

*Model - Patient

Thuộc tính:

id

name

code

phonenumber

Vai trò: đối tượng dữ liệu bệnh nhân dùng xuyên suốt controller →

service → repository.

* Luồng xử lý chính trong sơ đồ

Client gửi yêu cầu qua API Gateway.

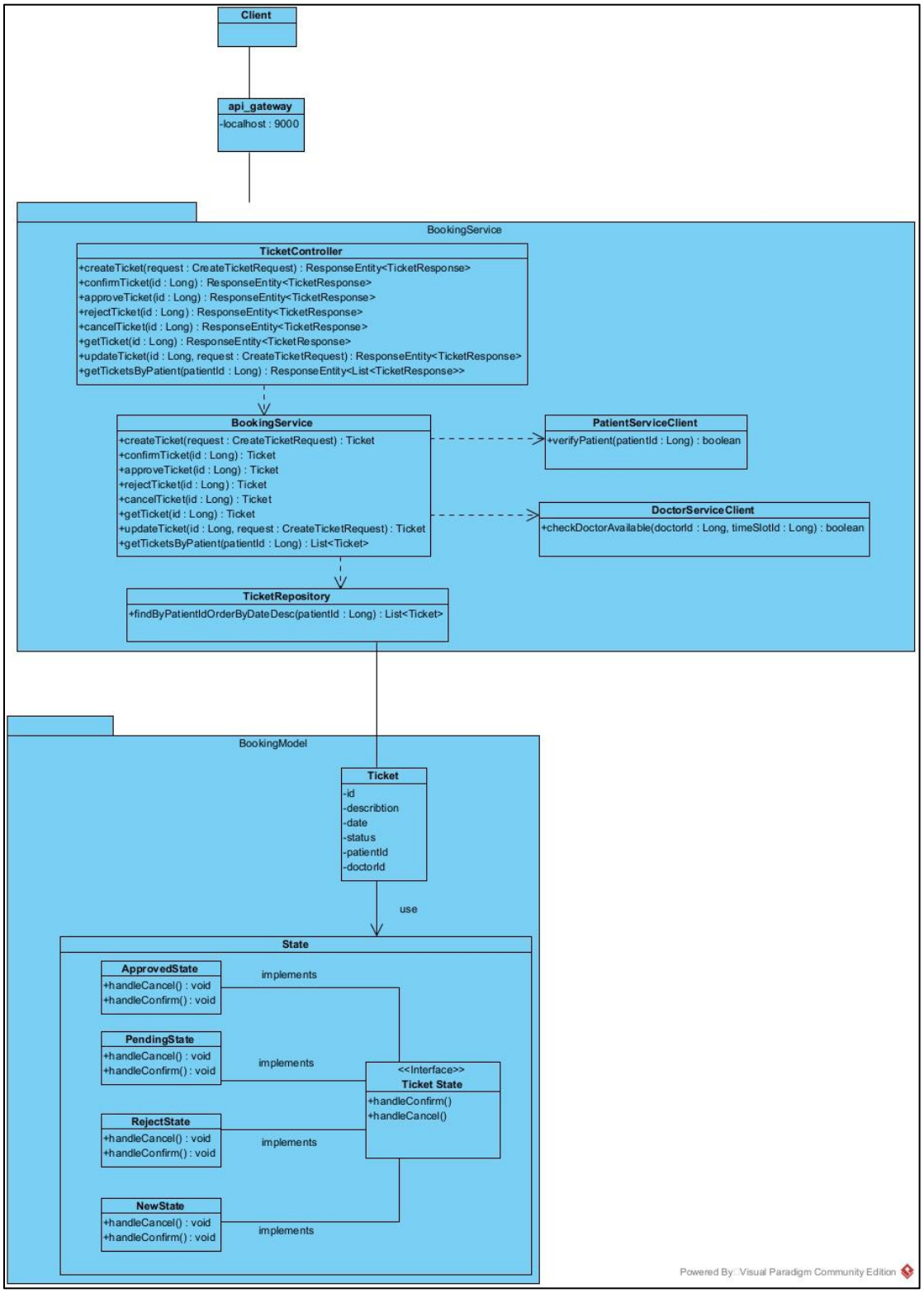
Gateway chuyển vào PatientController.

PatientController gọi PatientService.

PatientService gọi PatientRepository.

PatientRepository truy vấn Patient từ dữ liệu lưu trữ và trả ngược lên các tầng trên.

iii. BookingService



Booking Service là phần xử lý nghiệp vụ đặt lịch khám trong hệ thống. Nó nhận dữ liệu đặt lịch từ người dùng, tạo phiếu khám, kiểm tra trạng

thái hợp lệ của phiếu, cho phép xác nhận, duyệt, từ chối, hủy và tra cứu lịch khám của bệnh nhân. Nói ngắn gọn, đây là nơi quản lý toàn bộ vòng đời của một phiếu khám.

*Client: Là giao diện người dùng gửi yêu cầu đặt lịch khám, xem lịch sử phiếu khám, cập nhật hoặc hủy phiếu khám.

Client không truy cập trực tiếp BookingService, mà đi qua API Gateway.

*API Gateway (localhost:9000)

Là điểm vào duy nhất của request.

Nhận request từ Client rồi định tuyến đến BookingService.

*BookingService (server-side)

Bao gồm các lớp chính theo kiến trúc phân tầng:

- TicketController:

createTicket(request): ResponseEntity<TicketResponse>

confirmTicket(id: Long): ResponseEntity<TicketResponse>

approveTicket(id: Long): ResponseEntity<TicketResponse>

rejectTicket(id: Long): ResponseEntity<TicketResponse>

cancelTicket(id: Long): ResponseEntity<TicketResponse>

getTicket(id: Long): ResponseEntity<TicketResponse>

getTicketsByPatient(patientId: Long):

ResponseEntity<List<TicketResponse>>

updateTicket(id: Long, request): ResponseEntity<TicketResponse>

Vai trò: nhận request HTTP, gọi xuống service và trả kết quả cho client.

- BookingService:

createTicket(request): Ticket

confirmTicket(id: Long): Ticket

approveTicket(id: Long): Ticket

rejectTicket(id: Long): Ticket

cancelTicket(id: Long): Ticket

getTicket(id: Long): Ticket

getTicketsByPatient(patientId: Long): List<Ticket>

updateTicket(id: Long, request): Ticket

Vai trò: xử lý nghiệp vụ đặt lịch, kiểm tra dữ liệu bệnh nhân/bác sĩ qua client tích hợp, điều phối vòng đời trạng thái của Ticket.

- TicketRepository:

findById(id: Long): Optional<Ticket>

findByPatientIdOrderByDateDesc(patientId: Long): List<Ticket>

Vai trò: truy vấn dữ liệu phiếu khám trong database.

*Integration Client trong BookingService

- PatientServiceClient: verifyPatient(patientId: Long): boolean

Vai trò: kiểm tra bệnh nhân có tồn tại trước khi tạo/cập nhật phiếu khám.

- DoctorServiceClient: checkDoctorAvailable(doctorId: Long, timeSlotId: Long): boolean

Vai trò: kiểm tra bác sĩ có trống theo khung giờ trước khi đặt lịch.

*Model - Ticket

Thuộc tính:

id

patientId

doctorId

name

phone

description

date

timeSlotId

state

Vai trò: đối tượng dữ liệu phiếu khám dùng xuyên suốt controller → service → repository.

*State pattern - TicketState

- Gồm các trạng thái:

NEW: phiếu vừa tạo khi người dùng xác nhận lần đầu, đợi người dùng xác nhận lần cuối để tạo phiếu

PENDING: đã xác nhận lần cuối, chờ duyệt

APPROVED: đã duyệt

REJECTED: bị từ chối hoặc hủy

- Vai trò: quản lý chuyển trạng thái của phiếu khám theo đúng nghiệp vụ, giúp tách logic trạng thái khỏi lớp service.

- Ý nghĩa: Nếu không dùng State Pattern, BookingService sẽ phải viết kiểu:

nếu state = NEW thì confirm được

nếu state = PENDING thì approve/reject được

nếu state = APPROVED thì không làm gì thêm

nếu state = REJECTED thì không làm gì thêm

Càng thêm trạng thái mới, BookingService càng phình to, dễ lỗi nhánh logic.

State Pattern chuyển phần đó vào từng class state, nên BookingService chỉ cần gọi: ticket.requestConfirm(), ticket.approve(), ticket.reject(), ticket.cancel()

- Lợi ích thực tế cho service này

Tách trách nhiệm rõ: BookingService lo orchestration, state class lo luật chuyển trạng thái.

Dễ bảo trì: sửa luật của PENDING thì chỉ sửa PendingState.

Dễ mở rộng: thêm trạng thái mới không phải viết lại nhiều nhánh ở service.

*Luồng xử lý chính

Client gửi yêu cầu qua API Gateway.

Gateway chuyển vào TicketController.

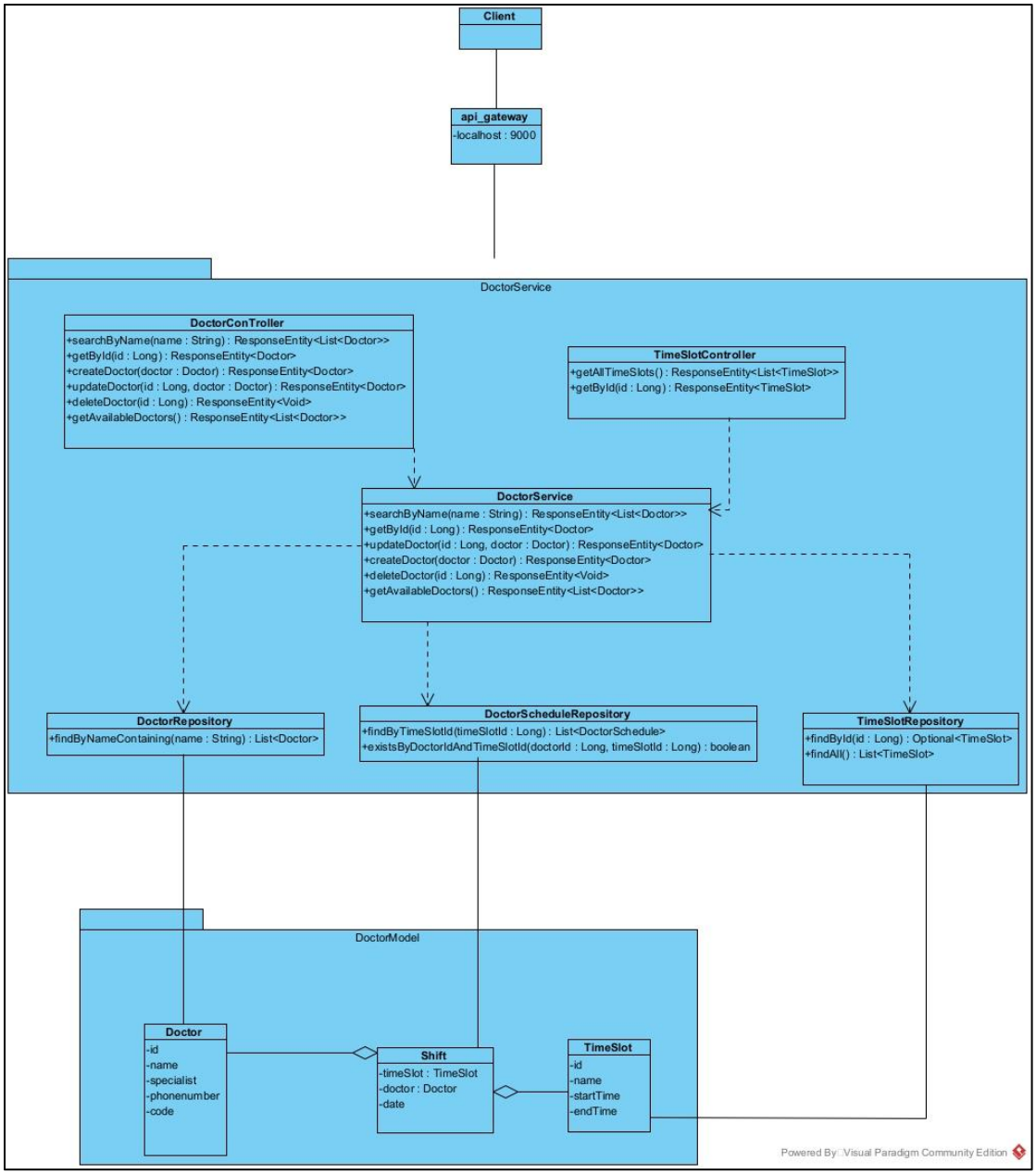
TicketController gọi BookingService.

BookingService gọi PatientServiceClient và DoctorServiceClient để kiểm tra dữ liệu liên quan.

BookingService gọi TicketRepository để lưu/truy vấn Ticket.

TicketRepository trả kết quả ngược lên các tầng trên và trả response cho client.

iv. DoctorService



Diễn giải:

Doctor Service giữ vai trò cung cấp và quản lý dữ liệu bác sĩ, ca khám và lịch trực để phục vụ đặt lịch khám.Trong luồng đặt lịch, service này trả về danh sách ca khám, danh sách bác sĩ theo ca và kiểm tra bác sĩ có sẵn ở ca được chọn hay không. Đây là service đảm bảo người dùng chỉ đặt được lịch vào bác sĩ và khung giờ hợp lệ, giúp Booking Service ra quyết định tạo lịch chính xác.

*Client: Là giao diện người dùng hoặc service gọi đến để lấy danh sách bác sĩ, ca khám và kiểm tra bác sĩ còn trống khi đặt lịch.

Client không truy cập trực tiếp DoctorService, mà đi qua API Gateway.

*API Gateway (localhost:9000)
Là điểm vào duy nhất của request.
Nhận request từ Client rồi định tuyến đến DoctorService.

*DoctorService (server-side)
Bao gồm các lớp theo kiến trúc phân tầng:
- DoctorController:
searchByName(name: String): ResponseEntity<List<Doctor>>
getById(id: Long): ResponseEntity<Doctor>
createDoctor(doctor: Doctor): ResponseEntity<Doctor>
updateDoctor(id: Long, doctor: Doctor): ResponseEntity<Doctor>
deleteDoctor(id: Long): ResponseEntity<Void>
getAvailableDoctors(): ResponseEntity<List<Doctor>>
Vai trò: nhận request HTTP liên quan bác sĩ, gọi xuống service và trả kết quả cho client.

- TimeSlotController:
getAllTimeSlots(): ResponseEntity<List<TimeSlot>>
getById(id: Long): ResponseEntity<TimeSlot>
Vai trò: nhận request HTTP liên quan ca khám, gọi service để trả dữ liệu ca khám.

- DoctorService:
searchByName(name: String): List<Doctor>
getById(id: Long): Doctor
createDoctor(doctor: Doctor): Doctor
updateDoctor(id: Long, doctor: Doctor): Doctor

deleteDoctor(id: Long): void
getAvailableDoctors(): List<Doctor>
Vai trò: xử lý nghiệp vụ trung gian, kiểm tra dữ liệu, điều phối truy vấn lịch trực và tình trạng sẵn sàng của bác sĩ trước khi trả về controller.

- DoctorRepository
findByNameContaining(name: String): List<Doctor>
Vai trò: truy vấn dữ liệu bác sĩ theo tên và các thông tin bác sĩ trong database.

- DoctorScheduleRepository
findByTimeSlotId(timeSlotId: Long): List<DoctorSchedule>
existsByDoctorIdAndTimeSlotId(doctorId: Long, timeSlotId: Long): boolean
Vai trò: quản lý dữ liệu phân ca bác sĩ theo ca khám.

- TimeSlotRepository
findById(id: Long): Optional<TimeSlot>
findAll(): List<TimeSlot>
Vai trò: truy vấn dữ liệu ca khám trong database.

*Model - DoctorModel
Gồm các thực thể chính:
- Doctor: id, name, specialization, phonenumber, code
Vai trò: đối tượng dữ liệu bác sĩ dùng xuyên suốt controller -> service -> repository.

- TimeSlot: id, name, startTime, endTime
Vai trò: biểu diễn khung giờ khám để bệnh nhân chọn khi đặt lịch.

Shift: doctor, timeSlot, date
Vai trò: thực thể liên kết bác sĩ với ca khám, giúp xác định bác sĩ nào làm ở ca nào.

*Luồng xử lý chính trong sơ đồ:
Client gửi yêu cầu qua API Gateway.
Gateway chuyển vào DoctorController hoặc TimeSlotController.
Controller gọi DoctorService.
DoctorService gọi DoctorRepository, DoctorScheduleRepository và TimeSlotRepository.
Repository truy vấn dữ liệu bác sĩ, ca khám, phân ca và trả ngược lên service. Service xử lý nghiệp vụ rồi trả kết quả lên controller để phản hồi về client.

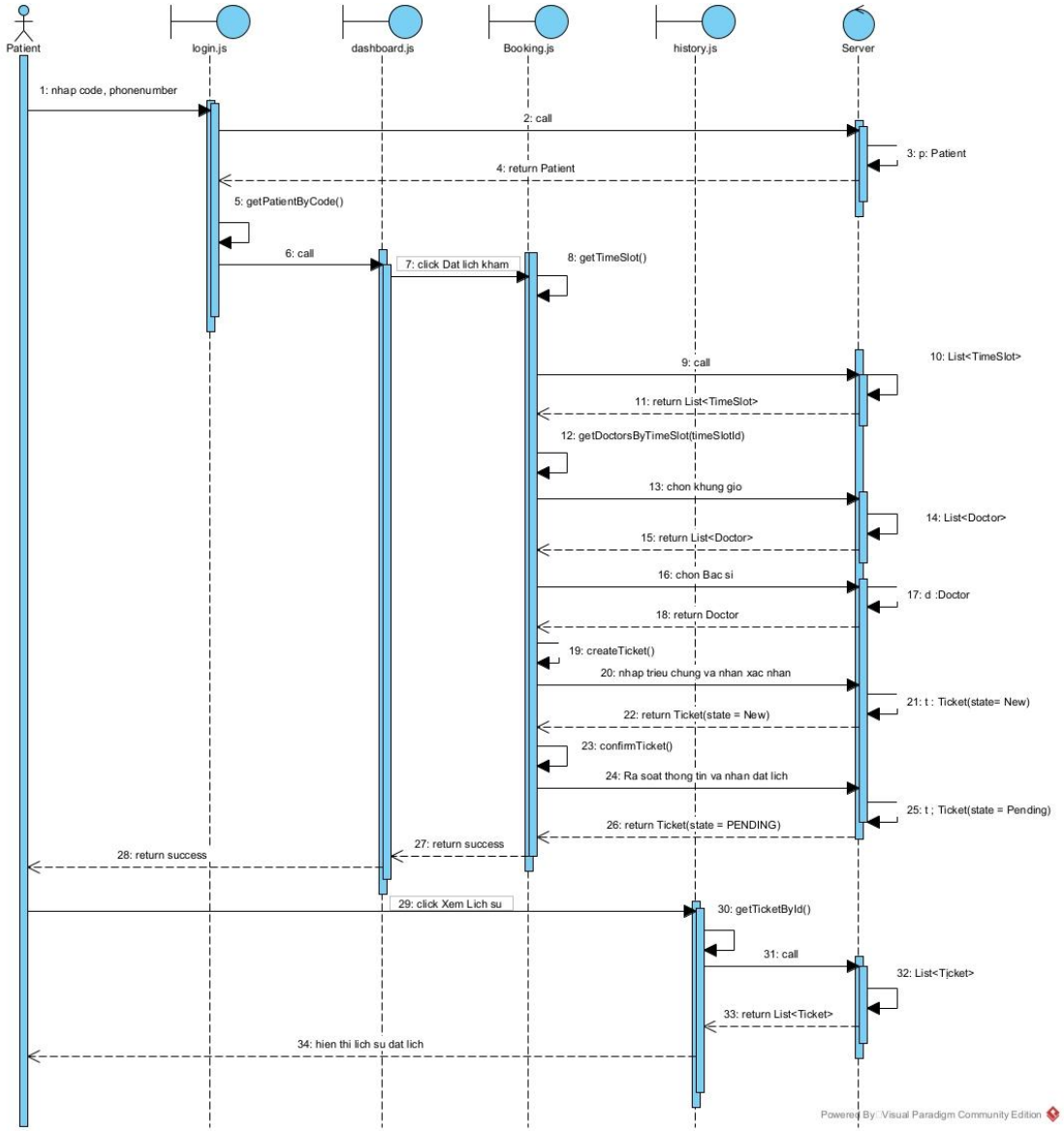
iii. Thiết kế biểu đồ tuần tự hoạt động chi tiết cho mỗi bên/ứng dụng

Kịch bản chuẩn

1. Bệnh nhân đăng nhập thành công vào hệ thống với username và password lần lượt là mã bệnh nhân và số điện thoại.
2. Giao diện dashboard hiện ra gồm nút "Đặt lịch khám" và "Lịch sử khám".
3. Bệnh nhân click vào nút "Đặt lịch khám".
4. Giao diện đặt lịch hiện ra gồm trường: ngày khám
5. Bệnh nhân chọn ngày khám phù hợp.
6. Giao diện hiển thị các khung giờ có thể đặt lịch của ngày đó.
7. Bệnh nhân chọn khung giờ khám.
8. Giao diện hiển thị danh sách bác sĩ còn trống theo khung giờ đã chọn.

9. Bệnh nhân chọn bác sĩ mong muốn
10. Giao diện hiển thị ô nhập "Mô tả triệu chứng"
11. Bệnh nhân nhập mô tả triệu chứng và bấm Xác nhận đặt lịch.
12. Giao diện Xác nhận hiện ra gồm các thông tin bệnh nhân vừa nhập vào phiếu và nút "Đặt lịch"
13. Bệnh nhân rà soát lại thông tin và bấm "Đặt lịch"
14. Hệ thống thông báo "Đặt lịch thành công" .
15. Bệnh nhân có thể xem lại các phiếu đặt lịch của mình và trạng thái ở trong mục "Lịch sử khám".

1. Server là hộp đen



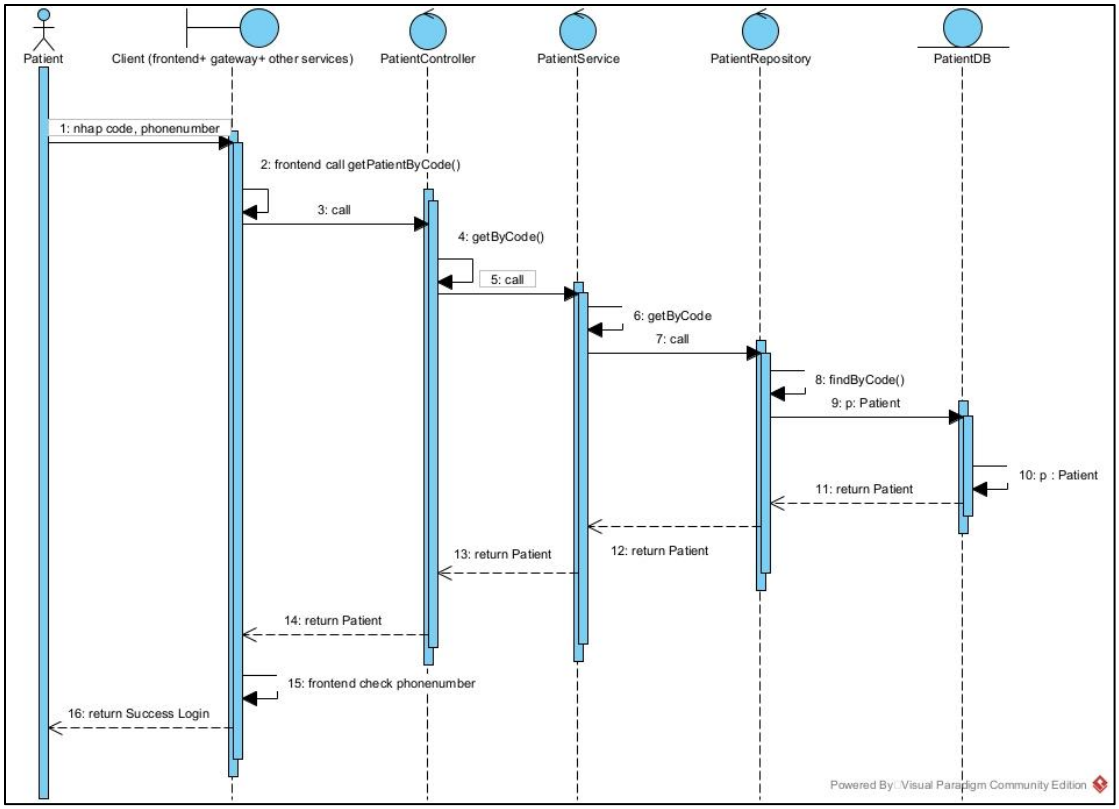
1. Người dùng nhập mã username(code) và password (số điện thoại) trên giao diện đăng nhập.
2. login.js gửi yêu cầu call đến Server.
3. Server thực hiện xử lý tìm kiếm và trả về đối tượng p: Patient.
4. Server trả thông tin Patient về cho login.js.
5. login.js tự thực hiện hàm nội bộ getPatientByCode().
6. login.js gọi chuyển hướng sang giao diện dashboard.js.
7. Patient thực hiện click vào chức năng "Đặt lịch khám" trên dashboard.js.
8. dashboard.js chuyển yêu cầu sang Booking.js, đồng thời Booking.js gọi hàm getTimeSlot().

9. Booking.js gửi yêu cầu call đến Server để lấy danh sách khung giờ.
10. Server truy vấn và lấy ra List<TimeSlot>.
11. Server trả về danh sách khung giờ (List<TimeSlot>) cho Booking.js để hiển thị cho người dùng.
12. Booking.js gọi hàm nội bộ getDoctorsByTimeSlot(timeSlotId) để chuẩn bị hiển thị bác sĩ.
13. Người dùng nhấn nút chọn khung giờ .
14. Server xử lý và trả về danh sách bác sĩ tương ứng (List<Doctor>).
15. Server trả kết quả List<Doctor> về cho Booking.js.
16. Người dùng nhấn vào nút "chọn Bác sĩ" .
17. Server xác nhận thông tin bác sĩ và trả về đối tượng d: Doctor.
18. Server phản hồi thông tin Doctor về cho Booking.js.
19. Booking.js tự gọi hàm khởi tạo createTicket().
20. Patient nhập triệu chứng và nhấn xác nhận trên giao diện, Booking.js gửi yêu cầu này đến Server.
21. Server tạo ticket mới t: Ticket (state = New).
22. Server trả Ticket trạng thái "New" về cho Booking.js.

23. Booking.js tự thực hiện hàm confirmTicket(), hiển thị giao diện xác nhận lần cuối là Ticket(state =NEW) gồm các thông tin về khung giờ, bác sĩ, triệu chứng người dùng đã chọn.
24. Người dùng rà soát thông tin lần cuối và nhấn "Đặt lịch" (xác nhận cuối cùng) .
25. Server cập nhật trạng thái Ticket thành t: Ticket (state = Pending).
26. Server trả Ticket trạng thái "Pending" về cho Booking.js.
27. Booking.js phản hồi thông báo thành công (return success) về cho dashboard.js.
28. dashboard.js hiển thị thông báo thành công cho người dùng.
29. Người dùng click vào chức năng "Xem Lịch sử" trên giao diện.
30. history.js nhận yêu cầu và gọi hàm getTicketById().
31. history.js gửi yêu cầu call đến Server.
32. Server truy vấn và lấy ra danh sách các vé khám (List<Ticket>).
33. Server trả danh sách List<Ticket> về cho history.js.
34. history.js thực hiện hiển thị lịch sử đặt lịch lên màn hình cho người dùng.

2. Client là hộp đen

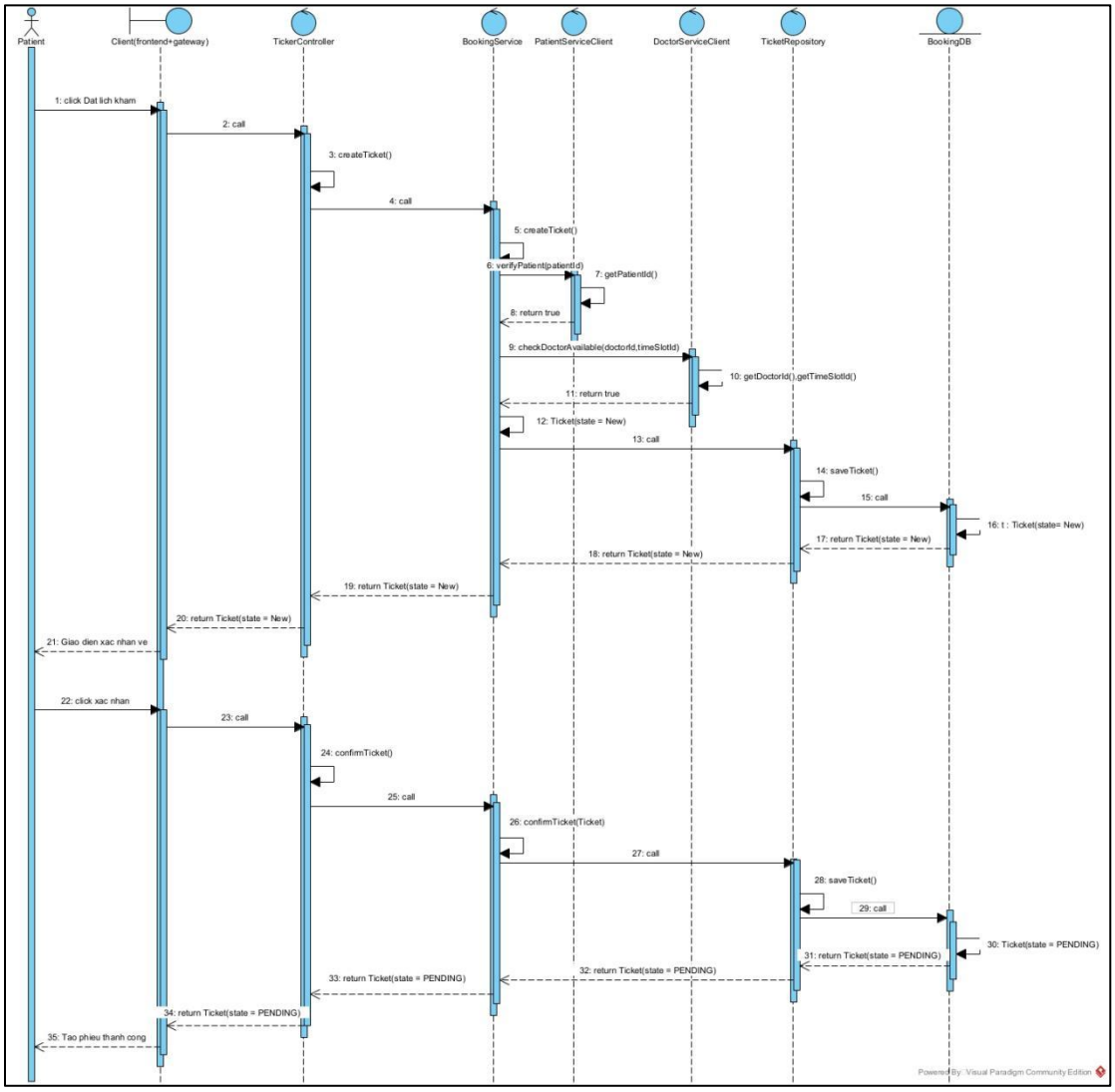
a) Patient Service (Đăng nhập với username và password lần lượt là mã và số điện thoại)



1. Patient nhập code và phone number vào hệ thống.
2. Client (frontend) gọi API getPatientByCode().
3. Client gửi request đến PatientController.
4. PatientController xử lý và gọi hàm getByCode().
5. PatientController gọi sang PatientService.
6. PatientService tiếp nhận và xử lý getByCode.
7. PatientService gọi đến PatientRepository.
8. PatientRepository thực hiện findByCode().
9. PatientRepository truy vấn dữ liệu từ PatientDB.
10. PatientDB trả về thông tin bệnh nhân (Patient).
11. PatientRepository trả dữ liệu bệnh nhân về PatientService.
12. PatientService trả dữ liệu về PatientController.
13. PatientController trả dữ liệu bệnh nhân về Client.
14. Client nhận dữ liệu bệnh nhân.
15. Client (frontend) kiểm tra phonenumber với dữ liệu nhận được .

16. Nếu hợp lệ, trả về kết quả đăng nhập thành công (Success Login) cho Patient.

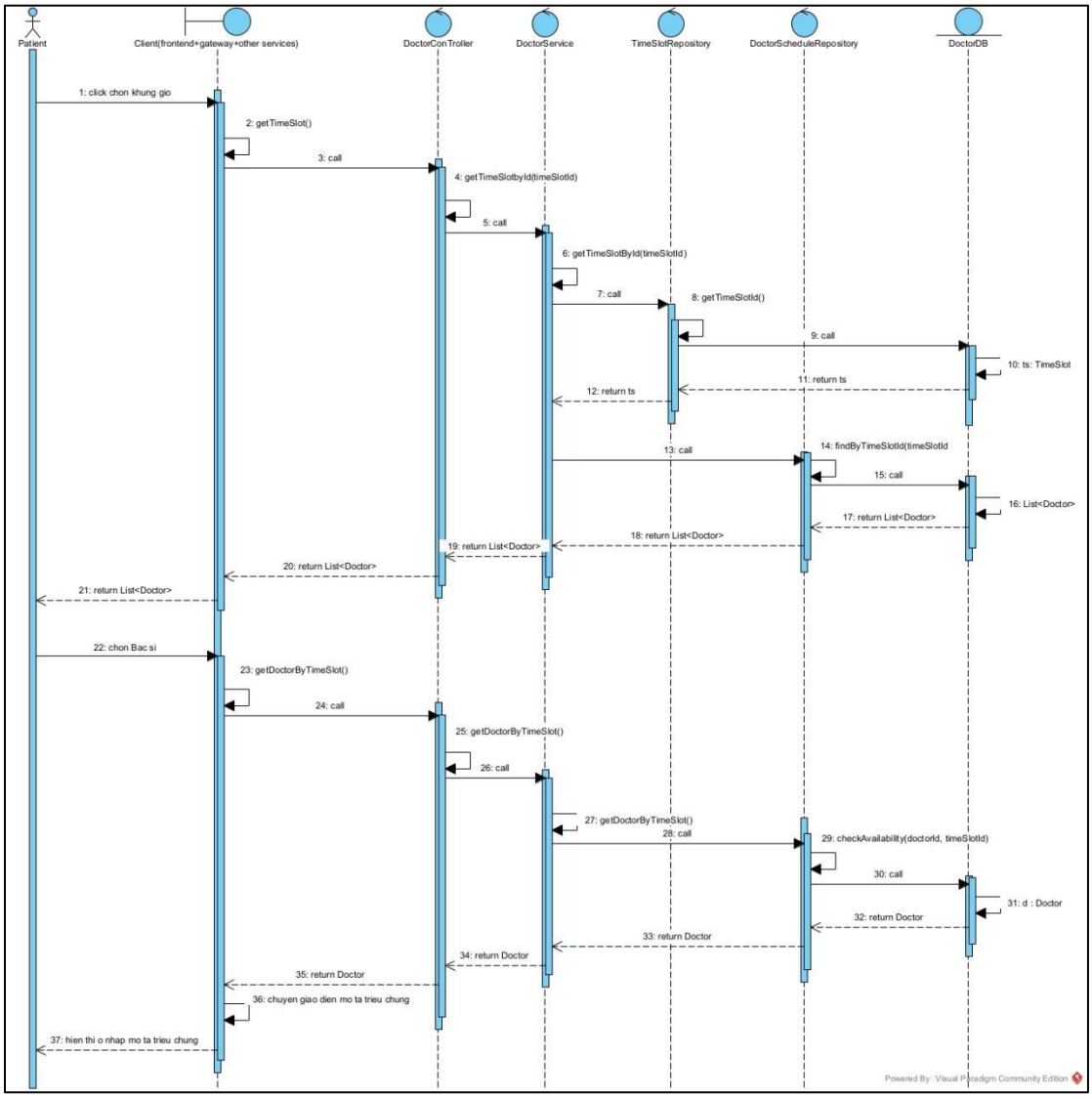
b) Booking Service (Đặt lịch khám)



1. Patient click chức năng Đặt lịch khám.
2. Client gửi request đến TicketController.
3. TicketController gọi hàm createTicket().
4. TicketController gọi sang BookingService.
5. BookingService bắt đầu xử lý createTicket().
6. BookingService gọi PatientServiceClient để xác thực bệnh nhân (verifyPatient).
7. PatientServiceClient gọi getPatientId().
8. Trả về kết quả xác thực hợp lệ (true).

9. BookingService gọi DoctorServiceClient để kiểm tra lịch bác sĩ (checkDoctorAvailable).
10. DoctorServiceClient lấy doctorId và timeSlotId.
11. Trả về kết quả bác sĩ có lịch trống (true).
12. BookingService tạo Ticket với trạng thái ban đầu = New.
13. BookingService gọi TicketRepository.
14. TicketRepository thực hiện saveTicket().
15. TicketRepository gọi đến BookingDB để lưu dữ liệu.
16. BookingDB lưu Ticket với trạng thái = New.
17. BookingDB trả về Ticket đã lưu.
18. TicketRepository trả Ticket về BookingService.
19. BookingService trả Ticket về TicketController.
20. TicketController trả Ticket về Client.
21. Client hiển thị giao diện xác nhận vé cho Patient.
22. Patient click xác nhận đặt lịch.
23. Client gửi request đến TicketController.
24. TicketController gọi hàm confirmTicket().
25. TicketController gọi sang BookingService.
26. BookingService xử lý confirmTicket(Ticket).
27. BookingService gọi TicketRepository.
28. TicketRepository thực hiện saveTicket() lần nữa để cập nhật.
29. TicketRepository gọi đến BookingDB.
30. BookingDB cập nhật Ticket với trạng thái = PENDING.
31. BookingDB trả về Ticket đã cập nhật.
32. TicketRepository trả Ticket về BookingService.
33. BookingService trả Ticket về TicketController.
34. TicketController trả Ticket về Client.
35. Client thông báo tạo phiếu khám thành công cho Patient.

c) Doctor Service(chọn khung giờ và bác sĩ làm việc trong khung giờ đó)



1. Patient click chọn khung giờ trên giao diện.
2. Client gửi request getTimeSlot() để xử lý.
3. Client gọi sang DoctorController.
4. DoctorController gọi hàm getTimeSlotById(timeSlotId).
5. DoctorController gọi sang DoctorService.
6. DoctorService xử lý hàm getTimeSlotById(timeSlotId).
7. DoctorService gọi sang TimeSlotRepository.
8. TimeSlotRepository thực hiện hàm getTimeSlot().

9. TimeSlotRepository gọi đến DoctorDB để truy vấn dữ liệu.
10. DoctorDB tìm và trả về đối tượng ts: TimeSlot.
11. DoctorDB phản hồi kết quả về cho TimeSlotRepository.
12. TimeSlotRepository trả kết quả ts về cho DoctorService.
13. DoctorService tiếp tục gọi sang DoctorScheduleRepository.
14. DoctorScheduleRepository thực hiện hàm findByTimeSlotId(timeSlotId).
15. DoctorScheduleRepository gọi đến DoctorDB để lấy danh sách bác sĩ.
16. DoctorDB trả về danh sách bác sĩ List<Doctor>.
17. DoctorDB phản hồi kết quả về cho DoctorScheduleRepository.
18. DoctorScheduleRepository trả danh sách List<Doctor> về cho DoctorService.
19. DoctorService trả danh sách List<Doctor> về cho DoctorController.
20. DoctorController trả danh sách List<Doctor> về cho Client.
21. Client hiển thị danh sách bác sĩ tương ứng với khung giờ cho Patient.
22. Patient thực hiện chọn một bác sĩ cụ thể từ danh sách (chon Bac si).
23. Client gửi request getDoctorByTimeSlot() đến DoctorController.
24. DoctorController gọi hàm xử lý tương ứng.
25. DoctorController gọi sang DoctorService để xử lý getDoctorByTimeSlot().
26. DoctorService tiếp nhận yêu cầu và xử lý logic.
27. DoctorService gọi hàm getDoctorByTimeSlot() sang lớp Repository.
28. DoctorService gọi sang DoctorScheduleRepository.
29. DoctorScheduleRepository thực hiện hàm checkAvailability(doctorId, timeSlotId).
30. DoctorScheduleRepository gọi đến DoctorDB để kiểm tra tính khả dụng.
31. DoctorDB trả về thông tin đối tượng d: Doctor.

32. DoctorDB phản hồi kết quả về cho DoctorScheduleRepository.
33. DoctorScheduleRepository trả đối tượng Doctor về cho DoctorService.
34. DoctorService trả đối tượng Doctor về cho DoctorController.
35. DoctorController trả đối tượng Doctor về cho Client.
36. Client thực hiện chuyển sang giao diện nhập mô tả triệu chứng (.
37. Client hiển thị ô nhập mô tả triệu chứng cho Patient.